QZPaper Huggingface Daily

2026-03-04

---

1. **SWE-rebench V2：大规模语言无关的软件工程任务收集与训练环境构建**
   SWE-rebench V2 构建首个语言无关的大规模软件工程训练环境，通过五阶段流水线从 GitHub PR 自动挖掘任务、合成 Docker 容器环境、提取测试用例，产出 32000+ 可执行任务覆盖 20 种编程语言，为强化学习训练提供海量高质量数据。
2. **LK Losses：面向投机解码的直接接受率优化**
   LK Losses 论证投机解码中最大化接受率严格等价于最小化 Total Variation 距离而非 KL 散度，提出稳定的 TV-based 损失函数，克服直接 TV 优化的梯度不稳定问题，显著提升草稿模型的接受率。
3. **From Scale to Speed：面向图像编辑的自适应测试时缩放**
   From Scale to Speed 为图像编辑提出自适应测试时缩放框架 ADE-CoT，通过难度感知的动态采样、可靠评估和智能去重，在保持编辑质量的前提下显著降低计算成本，将焦点从规模转向速度。
4. **CHIMERA：面向可泛化 LLM 推理的紧凑合成数据**
   CHIMERA 建立全自动 LLM 驱动的推理数据合成流水线，包含主题扩展、问题生成和解答合成三阶段，覆盖 1179 个细粒度主题，通过跨模型验证和多样解答生成，为可泛化推理提供大规模高质量训练数据。
5. **VGGT-Det：挖掘 VGGT 内部先验实现无传感器几何的多视角室内 3D 目标检测**
   VGGT-Det 提出首个无传感器几何的多视角 3D 目标检测框架，深度挖掘预训练 VGGT 编码器的内部语义和几何先验，通过注意力引导查询生成和查询驱动特征聚合两个策略，实现仅从无位姿图像的 3D 检测。
6. **RubricBench：对齐模型生成评分标准与人类标准**
   RubricBench 构建含 1147 个成对比较的基准，采用三阶段流水线标注原子化的人类评分标准，首次全面评估 rubric 引导评估的可靠性，填补模型生成 rubric 与人类标准对齐度量的空白。
7. **OpenAutoNLU：面向自然语言理解的开源 AutoML 库**
   OpenAutoNLU 为文本分类和 NER 设计专门的开源 AutoML 库，核心创新是数据感知的训练方案自动选择，同时集成数据质量诊断、无监督 OOD 检测和 LLM 增强，实现从数据加载到模型导出的完整自动化。
8. **CoVe：基于约束引导验证的交互式工具使用智能体训练**
   CoVe 通过显式任务约束的多层处理，构建交互式工具使用智能体的高质量训练数据，约束经模糊化指导轨迹生成，原始约束作为确定性验证器，保证数据复杂度和轨迹正确性。
9. **MMR-Life：面向真实生活场景的多模态多图推理基准**
   MMR-Life 首个全面评估 MLLM 真实生活多图推理的基准，包含 2646 题和 19108 张真实图片，覆盖 7 种推理类型和 21 个任务，平均每题 7.22 张图片，系统评估 MLLMs 在日常推理中的能力差距。
10. **OmniLottie：基于参数化 Lottie Token 的矢量动画生成**
    OmniLottie 设计 Lottie Tokenizer 将冗余 JSON 压缩 5.3 倍为命令-参数序列，基于 Qwen2.5-VL 构建自回归生成模型，支持文本/图文/视频三种输入，配合 MMLottie-2M 数据集实现端到端多模态矢量动画生成。

SWE-rebench V2：大规模语言无关的软件工程任务收集与训练环境构建

Badertdinov, Nekrashevich, Shevtsov, Golubev — 2025

1. 动机与问题定义

软件工程智能体（SWE Agent）正在快速进步，近期提升主要由**强化学习（RL）**驱动。然而，RL 训练受限于大规模、具备可复现执行环境和可靠测试套件的任务集的稀缺性。构建此类环境面临三重困难：

- **依赖安装**：异构构建系统（Maven、Cargo、Go modules、npm 等）导致跨语言环境配置极为复杂；
- **测试可复现性**：测试运行器、输出格式、报告机制因生态系统而异，难以统一解析；
- **规范与测试对齐**：自然语言问题描述与测试 oracle 之间常存在语义错位。

现有基准如 SWE-bench 及其变体主要面向**评估**而非训练，且多集中于 Python 单一生态。多语言基准（Multi-SWE-bench、SWE-PolyBench）虽扩展了语言覆盖，但构建成本高、规模有限。自动化流水线（SWE-rebench V1、SetUpAgent、SWE-Factory）在可扩展性上取得进展，但两大障碍仍存：(1) 语言无关性在实践中的鲁棒性未经充分验证；(2) 多数资源面向评估而非交互式学习，缺少预构建环境和实例级诊断元数据等训练导向制品。

SWE-rebench V2 是首个**语言无关的大规模训练环境构建流水线**，通过统一构建工作流配合少量可复用的语言特定模板（基础镜像、运行器、解析器），从 GitHub PR 历史中挖掘任务、构建容器化环境、提取 fail-to-pass 测试，并通过自动化质量评估过滤歧义或无效任务。

核心产出	规模
容器化可执行任务	32,000+ 任务，3,600+ 仓库
覆盖语言	20 种编程语言
PR 派生扩展任务	120,000+ 任务
时间跨度	2014–2025

2. 流水线架构

流水线包含五个阶段，从原始 GitHub PR 到可执行训练环境：

2.1 阶段 1：初步数据收集

从 GitHub Archive 聚合公开活动，关联 issue 与 PR 元数据，克隆仓库提取 commit 补丁。通过分布式 map-reduce 作业直接从本地 git 历史提取补丁，绕过 GitHub API 速率限制。过滤条件包括：许可证（宽松开源）、已解决 issue 关联已合并 PR、PR 包含测试修改。对每个 PR，将 diff 拆分为**解决方案补丁**（非测试文件）和**测试补丁**（测试文件），测试文件通过正则 `(?i)(test(?:ing|s)?|e2e)` 识别。

对高资源语言（Python、Java、Go）采用严格阈值（≥25 星、≥15 已关闭 issue），因为约 20% 仓库即覆盖约 80% 任务；对长尾语言放宽为 10 星、1 issue 以保多样性。此阶段产出约 21,000 仓库、580,000 任务实例。

2.2 阶段 2：环境合成（Setup Synthesis）

为每个仓库构建正确的 Docker 环境。每种语言预构建基础镜像（含运行时和通用工具），大型生态提供多版本（如 JDK 11/17/21）。关键设计：**每仓库仅推断一次**安装和测试流程（基于最新任务对应的快照），然后复用于该仓库所有任务。

采用**交互式 Setup Agent**（基于 mini-SWE-agent v1.14.4 + Qwen3-Coder-480B）进行闭环调试：检查代码库 → 尝试安装依赖 → 根据构建错误和测试失败迭代改进脚本。对编译语言（C/C++），agent 显式插入重编译步骤。成功后重新运行以验证可复现性。

日志解析器自动生成：从成功执行的测试日志中，用 LLM 合成仓库特定解析器，将输出映射为标准化 schema（pass/fail/error + 测试标识符）。大多数生态系统每仓库仅需 1–2 个解析器。

2.3 阶段 3：执行验证

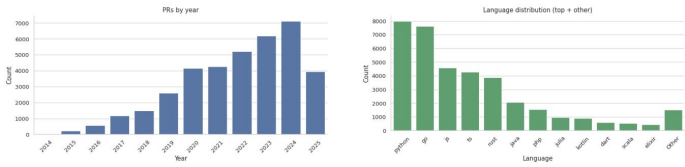
使用多阶段 Docker 构建分离基础层和仓库特定层。先应用测试补丁运行完整测试套件（pre-fix），再应用解决方案补丁重新运行（post-fix），产生配对执行轨迹。**始终运行完整测试套件**以检测意外副作用，仅保留至少一个 fail-to-pass 测试的实例。

2.4 阶段 4：Issue 清晰度过滤

使用三个独立 LLM 评审（gpt-oss-120b、GLM-4.7、DeepSeek-V3.2）评估 issue 文本规范性，仅当三者均判定规范充分时保留实例。此**共识策略**（consensus）优先保证精确率（precision = 0.88），有效过滤欠规范任务。

2.5 阶段 5：元数据标注

基于七个前沿模型在 300 任务子集上的诊断分析，开发元提示自动标注每个任务的潜在问题和特征（如 B1: 测试套件耦合、B2: 隐式命名要求、B3: 外部依赖、B4–B7 等），支持课程学习和子集选择。



(a) Issue years. (b) Languages.

图 1：数据集的时间分布与编程语言分布。*Python (21.6%)* 和 *Go (20.6%)* 占比最高。

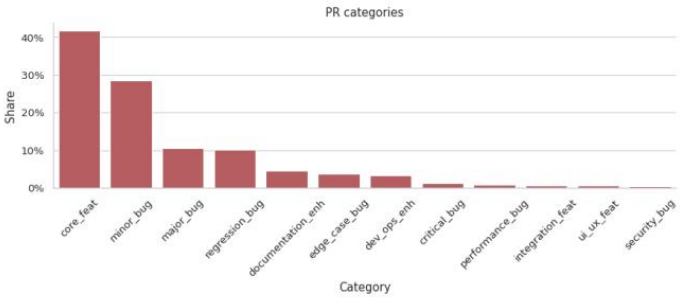
3. PR 派生任务扩展

Issue 关联是数据量的主要瓶颈（从 860 万含测试 PR 降至 80 万）。为此，对已成功收集任务的仓库，复用已有安装和测试指令，纳入**未关联 issue 的普通 PR**。对这些 PR，基于 PR 描述和补丁生成**合成问题描述**（而非直接使用 PR 文本），并通过后处理检测和过滤泄露解决方案的情况。此扩展额外释放 **120,000+** 任务。

4. 流水线漏斗

阶段	PRs	仓库
原始 PRs	29,511,758	145,306
含测试的 PRs	8,593,722	101,958
关联 issue + 测试	805,598	50,797
仓库级过滤后	583,809	21,692
成功执行 + F2P	41,349	4,006
Issue 文本过滤后	**32,079**	**3,617**

最终数据集中位任务修改 3 个文件、34 行代码（中等难度），但分布具有重尾性（第 90 百分位达 9 文件 / 181 行）。任务类型覆盖 12 类 PR：bug 修复、回归、文档、DevOps、性能、集成、UI/UX、安全等。



(a) Issue categories.

图 2：*Issue* 类型分布与补丁规模分布，展示任务类型多样性和难度重尾特征。

5. 消融实验

5.1 环境合成策略

在 103 个任务（10 种语言、每任务独立仓库）上对比不同策略，每配置独立运行 10 次：

配置	pass@1	pass@10
非交互式流水线	12.1	15.7
mini-SWE-agent (Qwen3-30B, 32k)	17.4	46.1
mini-SWE-agent (DeepSeek-V3.2, 32k)	20.3	59.8
mini-SWE-agent (Qwen3-480B, 32k)	25.8	58.8
mini-SWE-agent (Qwen3-480B, 128k)	27.1	**62.7**

关键发现：（1）**交互式系统一致优于非交互式**——即使 30B 交互式 agent 也超越 480B 非交互式流水线；（2）32k 上下文对大多数项目已足够，过长上下文可能导致 agent 陷入循环；（3）**多次尝试显著提升成功率**，从 1 次到 10 次近乎翻倍。实际流水线采用单次运行作为成本-产出折中。

5.2 Issue 清晰度过滤

在 SWE-bench Verified 标注集（1699 实例，3 名人类标注者）上消融：

Prompt 消融： Verified+ 获最佳 F1（0.50），而 Verified-E（同时提供 patch 和 test patch）获最高精确率（0.83），因过滤阶段优先精确率，流水线采用 Verified-E。

模型消融： gpt-oss-120b 综合表现最佳（Acc 0.68, Prec 0.75），Gemini 3 Pro 精确率最高（0.92）但召回极低（0.05），Claude Opus-4.5 精确率 0.83。

集成消融： 混合评审（gpt-oss-120b + GLM-4.7 + DeepSeek-V3.2）平均聚合 F1 最优（0.43），**共识聚合精确率达 0.88**，适合高精度过滤需求。

6. 诊断分析与任务质量

对 300 任务子集（Python/JS/Go/Rust/Scala 各 60）运行七个前沿模型（每模型 3 次独立运行），得到基线通过率：

模型	pass@1	pass@3
Claude Opus-4.5	25.2	32.7
GLM 4.7	21.3	26.7
MiniMax M2.1	19.2	27.0
Gemini 3 Flash	18.1	27.7
DeepSeek v3.2	17.4	25.0
GPT 5.2	17.0	25.0
gpt-oss-120b	8.8	14.3

Claude Opus-4.5 表现最优（pass@1 = 25.2%），Python 通过率最高（36.1%），Scala 最低（19.4%），体现语言间难度差异。

通过分析执行轨迹识别出三类系统性问题：

- **B1 测试套件耦合**：模型正确修复目标 issue 但因不相关代码路径回归而失败——这并非总是缺陷，常反映有效的回归检测；
- **B2 隐式命名要求**：测试期望问题描述中未指定的具体实现细节（如函数名），可通过注入命名提示缓解；
- **B3 外部依赖**：问题描述引用 agent 评估时可能不可访问的外部 URL。

基于诊断标签的元数据使研究者可灵活构建训练集：过滤 B 类任务创建“干净”子集用于 SFT/RL 热启动；后期引入含噪任务训练鲁棒性；针对 B3 任务训练具备网页浏览能力的 agent。

7. 与相关工作对比

- **vs. SWE-bench**：从单语言评估扩展至 20 语言训练，规模从 2,294 提升至 32,000+（含 120K PR 派生），提供预构建环境；
- **vs. Multi-SWE-bench**：避免昂贵人工验证，通过 LLM 集成自动过滤实现大规模收集，同时释放训练导向制品；
- **vs. SWE-Factory**：覆盖 20 语言（vs. 4），提供实例级诊断元数据和 PR 派生扩展；
- **vs. SWE-smith**：基于真实 issue 而非合成任务，保留真实世界的噪声、歧义和工具变异性；
- **vs. SWE-rebench V1**：从 Python 单一生态扩展至语言无关，增加交互式 Setup Agent、LLM 集成过滤、元数据标注。

8. 局限性与未来方向

- 自动化构建不可避免引入部分缺陷任务（如环境准备问题导致奖励噪声），通过诊断元数据缓解但未完全消除；
- 当前环境设计限于**单 Docker 容器**可复现的项目，无法覆盖需多服务、外部数据库或其他基础设施的复杂系统；
- 未包含使用不同过滤子集训练 agent 的消融实验——验证元数据对课程设计的有效性是未来工作；
- 单次 setup 尝试仅约 20% 仓库成功，增加重试次数和考虑多仓库状态（不同 commit/工具链版本）可提高产出；
- 未来计划：扩展至多服务长期任务、引入超越测试正确性的奖励信号（性能、延迟、内存效率等非功能性需求）、持续纳入更多长尾语言。

LK Losses：面向投机解码的直接接受率优化

Samarin, Krutikov, Shevtsov, Skvortsov, Fisin, Golubev — 2025

1. 问题动机与背景

大语言模型（LLM）推理受限于内存带宽而非计算吞吐量，自回归逐 token 生成的顺序内存访问严重低利用现代加速器。**投机解码**（Speculative Decoding）通过「草稿—验证」范式解决此问题：轻量级草稿模型（draft model）提出多个候选 token，目标模型（target model）在一次前向传播中并行验证，在保持目标模型输出分布不变的前提下实现显著加速。

加速效率的核心驱动因素是**接受率**（acceptance rate）α。然而，当前草稿模型训练普遍以 KL 散度作为代理目标。虽然 KL 散度与接受率共享相同的全局最优（$q = p$ 时 KL$= 0$, $\alpha = 1$），但草稿模型通常仅有目标模型 1–5% 的参数量，不可避免地收敛到次优解。**在次优点处，最小化 KL 散度并不保证最大化接受率**。

1.1 投机解码形式化

给定上下文 $\mathbf{c}$，草稿模型 q 提出 token 序列 $x_1, \ldots, x_K$，每个 token 的接受概率为：

$$\beta(x_i \mid \mathbf{c}, \mathbf{x}_{<i}) = \min\left(1,\ \frac{p(x_i \mid \mathbf{c}, \mathbf{x}_{<i})}{q(x_i \mid \mathbf{c}, \mathbf{x}_{<i})}\right).$$

第 i 个位置的期望接受率定义为：

$$\alpha_i = \sum_{x \in \mathcal{V}} \min\big(q(x \mid \mathbf{c}, \mathbf{x}_{<i}),\ p(x \mid \mathbf{c}, \mathbf{x}_{<i})\big).$$

关键性质：$\alpha = 1 - \mathrm{TV}(p, q)$，即**最大化接受率严格等价于最小化 Total Variation (TV) 距离**，而 KL 散度仅为代理目标。

1.2 动机示例

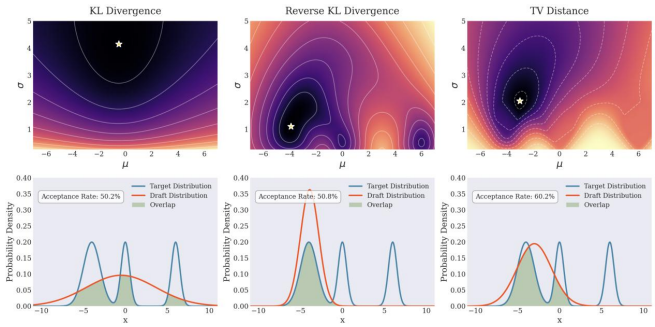


图 1：用单个高斯拟合混合高斯。*KL* 覆盖所有模态（$\alpha = 50.2\%$），反向 *KL* 坍缩到主模态（$\alpha = 50.8\%$），*TV* 最大化重叠（$\alpha = 60.2\%$）。

当草稿模型容量有限无法完美匹配目标分布时，**目标函数的选择决定了优化做出怎样的妥协**。前向 KL 因模式覆盖特性将概率质量分散，反向 KL 因模式寻找特性坍缩到单一模态，而 TV 距离找到分布重叠最大的解。

2. 散度损失的梯度分析

令 $q = \mathrm{softmax}(z_q)$，分析不同散度对 logits z_q 的梯度：

KL 散度梯度：

$$\nabla_{z_q} \mathrm{KL}(p \| q) = q - p.$$

梯度按预测与目标概率差值成比例推动每个 logit。在训练初期（q 近似均匀），梯度范数 $\|q - p\| = \mathcal{O}(1/\sqrt{k})$（$k$ 为目标分布支撑集大小），信号强且稳定。

TV 距离梯度：

$$\nabla_{z_q} \mathrm{TV}(p, q) = \tfrac{1}{2}\, q \odot \big(s - \mathbb{E}_q[s]\big),$$

其中 $s_i = \mathrm{sign}(q_i - p_i)$。存在三个严重问题：

- 仅依赖误差的**符号**而非幅度——轻微低估与严重低估获得相同信号；
- 损失面在 $\{z_q : q_i = p_i\}$ 处**不可微**，梯度不连续；
- 梯度范数 $\|\nabla \mathrm{TV}\| = \mathcal{O}(\sqrt{k}/V)$，词表 $V > 100$k 时**梯度消失**极其严重。

核心矛盾： KL 优化景观平滑但目标是代理；TV 直接优化正确目标但梯度病态。

3. 方法论：LK Losses

3.1 混合目标 $\mathcal{L}_{\mathrm{LK}}^{\lambda}$：自适应融合

$$\mathcal{L}_{\mathrm{LK}}^{\lambda}(p, q) = \lambda \cdot \mathrm{KL}(p \| q) + (1 - \lambda) \cdot \mathrm{TV}(p, q).$$

关键创新在于**自适应调度**——λ 根据当前接受率动态调整：

$$\lambda = \exp\big(-\eta \cdot \mathrm{sg}[\alpha]\big), \quad \eta > 0,$$

其中 sg[·] 为 stop-gradient。λ 对每个草稿位置独立计算。

行为特性： $\alpha \to 0$ 时 $\lambda \to 1$（KL 主导，提供平滑梯度导航）；$\alpha \to 1$ 时 $\lambda \to 0$（TV 主导，直接优化接受率）。可类比**信赖域方法**：

$$\min_q \mathrm{TV}(p, q) \quad \text{s.t.} \quad \mathrm{KL}(p \| q) \le \delta,$$

自适应调度隐式控制约束阈值 δ。消融实验证实固定权重 $\lambda = 0.5$ 效果显著劣于自适应调度。

3.2 似然方法 $\mathcal{L}_{\mathrm{LK}}^{\alpha}$：负对数接受率

将接受率 $\alpha = \sum_x q(x)\beta(x)$ 视为接受事件的边际概率，自然地取负对数似然：

$$\mathcal{L}_{\mathrm{LK}}^{\alpha}(p, q) = -\log \alpha = -\log \sum_{x \in \mathcal{V}} \min\big(p(x), q(x)\big).$$

梯度关系：

$$\nabla_{z_q} \mathcal{L}_{\mathrm{LK}}^{\alpha} = \frac{1}{\alpha} \nabla_{z_q} \mathrm{TV}(p, q).$$

$1/\alpha$ 因子在 α 小时自动放大梯度，解决 TV 的梯度消失问题。初始阶段梯度范数 $\|\nabla \mathcal{L}_{\mathrm{LK}}^{\alpha}\| = \mathcal{O}(1/\sqrt{k})$，与 KL 量级匹配，但梯度**方向**与 TV 一致。退化情况：当 p 为 one-hot 分布时，$\mathcal{L}_{\mathrm{LK}}^{\alpha}$ 退化为标准负对数似然 $-\log q(x^*)$。

3.3 词表截断的天然兼容性

EAGLE-3 采用截断词表（FR-Spec），将大量 token 的 q_i 置零。KL 训练中 $q_i = 0$ 且 $p_i > 0$ 导致 KL 发散至无穷，需将 p 重定义为 $\tilde{p} = \mathrm{softmax}(m \odot z_p)$，使 KL 成为「代理的代理」。而 LK losses 天然兼容：截断词表外的 token 对 α 贡献 $\min(p_i, 0) = 0$，无需修改 p，直接优化原始目标分布上的接受率。

4. 实验设置

目标模型（6 个）： Llama-3.1-8B-Instruct、Llama-3.3-70B-Instruct（稠密）；gpt-oss-20B、gpt-oss-120B、Qwen3-235B-A22B-Instruct、DeepSeek-V3-685B（MoE）。

草稿架构（4 种）： EAGLE-3（自回归 Transformer + 特征融合）、MEDUSA（并行解码头）、MLP Speculator（多阶段 MLP）、DeepSeek MTP（原生多 token 预测模块微调）。所有架构使用 $K = 6$ 个投机头。

训练： 660K 提示（Infinity-Instruct-0625），各目标模型生成响应。batch size 64，学习率 4×10^{-4}，余弦调度，AdamW（$\beta_1 = 0.9, \beta_2 = 0.95$），梯度裁剪 0.5。草稿模型从头训练 10 epochs（DeepSeek MTP 微调 1 epoch）。损失按位置指数衰减聚合（$\gamma = 0.8$）。

评估： MT-bench（对话）、HumanEval（代码）、GSM8K（数学），温度 $T = 0$ 和 $T = 1$，chain sampling。修复了 vLLM 中 greedy draft sampling 的 bug（原实现在非零温度下仍贪心采样草稿 token，导致接受率系统性偏低）。主指标为平均接受长度 τ。

5. 核心实验结果

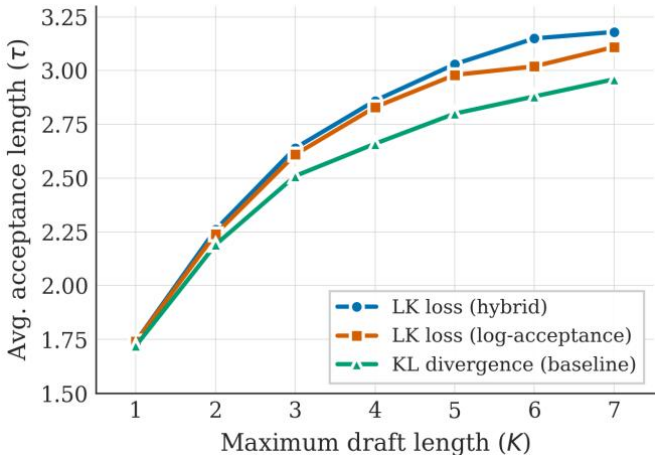


图 2: *EAGLE-3 + Qwen3-235B*，不同目标函数下 τ 随 K 的变化。*LK losses* 在所有 K 值上均优于 *KL*。

5.1 跨草稿架构（Llama-3.1-8B）

$T=1$ 下，EAGLE-3 + $\mathcal{L}_{\text{LK}}^{\lambda}$（$\eta=3$）：MT-bench 3.48（KL 3.39），HumanEval 4.52（KL 4.31），GSM8K 4.02（KL 3.88），平均提升 +3.8%。MEDUSA 平均提升 +7.8%（MT 1.85 vs 1.72，HE 2.22 vs 2.02，GSM 1.92 vs 1.81）。MLP Speculator 平均提升 +8.3%。**低容量草稿模型从直接接受率优化中获益更大**，与理论分析一致。

纯 TV 训练表现极差（EAGLE-3 $T=1$ MT-bench 仅 2.67），证实梯度病态使其不可行。固定 $\lambda=0.5$ 也显著劣于自适应调度，确认课程学习行为的必要性。

5.2 跨目标模型规模（8B–685B）

目标模型	KL $\bar{\tau}$	LK$^{\lambda}$ $\bar{\tau}$ (提升)
Llama-3.1-8B	3.86	4.01 (+3.9%)
Llama-3.3-70B	4.50	4.66 (+3.5%)
GPT-OSS 20B	3.17	3.29 (+3.8%)
GPT-OSS 120B	2.46	2.65 (+7.7%)
Qwen3-235B	3.77	4.08 (**+8.2%**)
DeepSeek-V3	4.43	4.68 (+5.6%)

$T=1$ 下所有模型均一致提升。最大增益出现在大型 MoE 模型配小型稠密草稿的场景：GPT-OSS 120B（+7.7%）、Qwen3-235B（+8.2%）。作者推测巨大的参数量差异和架构不匹配造成的容量鸿沟难以仅靠 KL 散度弥合。

DeepSeek-V3 的 MTP 模块原始权重仅训练预测下一个 token，自回归复用导致后续位置接受率急剧下降。KL 微调已大幅提升（$\bar{\tau}$: 3.09 → 4.43），LK loss 进一步推高至 4.68（+5.6%）。自适应 λ 调度器对此场景尤为合适：早期头接受率高获低 λ（TV 精调），后期头接受率低获高 λ（KL 引导）。

5.3 $T=0$（贪心解码）泛化

尽管训练目标在 $T=1$ 下优化，贪心解码下仍有一致提升，幅度稍小（Llama-8B EAGLE-3: +1.6%；Qwen3-235B: +1.8%）。

6. 消融实验

调度参数 η： $\eta=3$ 在多数场景最优；$\eta=1$ 时混合目标与似然目标差距缩小，说明较快的从 KL 到 TV 的过渡有利于性能。

$\mathcal{L}_{\text{LK}}^{\alpha}$ vs $\mathcal{L}_{\text{LK}}^{\lambda}$： 两者均优于 KL 基线。混合目标总体上略强于似然目标，尤其在贪心采样下；但似然目标实现更简单，无需调度超参。

对比 HuggingFace 公开检查点： 在相同评估流水线下，作者的 KL 基线已大幅超越公开检查点（例如 Llama-70B $T=1$: Ours KL 4.50 vs YH-70B 3.34），LK loss 进一步拉大差距。

7. 与相关工作对比

DistillSpec (Zhou et al., 2024) 探索了反向 KL 和 TV 作为预训练 LM 的知识蒸馏目标，但结论是「损失选择高度依赖任务和数据」。其设置为预训练模型微调，而非从头训练。本文从**梯度理论**出发揭示了 TV 在随机初始化下的根本缺陷，并通过自适应混合和似然方法系统性地解决。

EAGLE 系列在 KL 基础上加隐状态回归损失以缩小训练与推理差距；MEDUSA 混合 KL 与 LM 目标。这些仍以 KL 为核心，未直接优化接受率。

8. 讨论与局限

核心贡献： LK losses 是首个系统性的草稿模型直接接受率优化方法。两种变体（混合目标与似然目标）均零额外计算开销，可作为任何现有训练框架中 KL 损失的即插即用替换。

主要局限与未来方向：

- 当前优化每个位置的独立接受率，未直接优化系统效率（$(\tau-1)/K$，即接受 token 数与草稿长度之比）；
- 跨头损失聚合使用固定指数衰减 γ^{n-1}，可学习的逐头权重方案可能进一步提升；
- 部署时常用 top-k/top-p 采样参数，可纳入训练目标进行联合优化；
- 混合目标需调超参 η（实验表明 $\eta=3$ 普适性良好）。

From Scale to Speed：面向图像编辑的自适应测试时缩放

Qu[12*], Yuan[3*], Tang[3‡], Chen[3], Tang[3], Yu[3], Sun[3], Bai[3], Chu[3], Gou[12§], Xiong[12], Cai[4] — [1] 中科院信工所, [3] 阿里巴德, [4] 昆士兰大学, 2025

1. 动机与核心问题

Image Chain-of-Thought (Image-CoT) 是一种测试时缩放范式，通过延长推理时间、采样多个候选并择优选取来提升生成质量。现有 Image-CoT 研究主要聚焦文本到图像（T2I）生成，但图像编辑是**目标导向**任务——解空间受源图像和编辑指令约束，与 T2I 的开放式生成存在本质差异。直接将 T2I 的 Image-CoT 方法迁移至编辑任务暴露出三个关键问题：

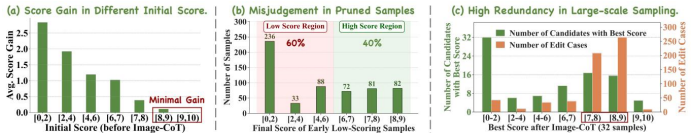


图 1：现有 *Image-CoT* 方法应用于编辑的三大问题。*(a)* 固定预算在简单编辑上浪费算力；*(b) 40%* 早期低分样本最终获得高分却被错误剪枝；*(c)* 大规模采样产生大量冗余正确结果。

- **资源分配低效**：现有方法对所有编辑使用固定采样预算（如 $N=32$）。但实验表明，简单编辑（初始分数高）几乎无法从 Image-CoT 获益，而困难编辑（初始分数低）收益显著。固定预算在简单案例上浪费大量算力。
- **早期验证不可靠**：现有方法依赖通用 MLLM 分数评估中间去噪状态进行早期剪枝。但编辑往往只修改源图像的局部细微区域，在早期去噪阶段难以区分。实验显示 **40%** 的早期低分样本最终达到高分，却被错误剪除。
- **编辑结果冗余**：大规模采样常产生多个具有相同最高分的正确结果。例如最佳分数在 $[7,9)$ 的编辑案例中，超过 15 个候选共享相同最高分。但编辑任务只需一个意图对齐的结果即可。现有广度优先剪枝策略导致不必要的计算开销。

针对上述问题，本文提出 **ADE-CoT**（ADaptive Edit-CoT），一个按需测试时缩放框架，将焦点从“规模”转向“速度”，通过三个核心策略在保持编辑正确性的同时显著提升效率。

2. 方法框架

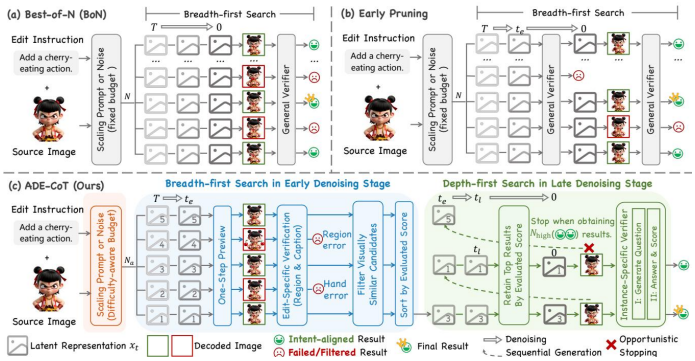


图 2：*Image-CoT* 方法流水线对比。*(a) BoN* 广度优先固定预算；*(b)* 通用分数剪枝；*(c) ADE-CoT* 三策略协同。

预备知识。给定源图像 I_{src} 和编辑指令 c，Best-of-N（BoN）方法生成 N 个候选 $\{I_1,\ldots,I_N\}$，由验证器 $\text{Vrf}: I_{\text{src}} \times I \times c \to \mathbb{R}$ 评分后选取最高分输出 $I^* = \arg\max_{I_i} \text{Vrf}(I_{\text{src}}, I_i, c)$。早期剪枝在时间步 $t_e < T$ 生成中间预览，低于拒绝阈值 S_{rj} 的候选被剪除。

2.1 难度感知资源分配

替代固定预算，ADE-CoT 根据编辑难度动态调整采样数。先生成单个候选并评估得到初始分数 S 作为难度代理，自适应预算为：

$$N_a = N_{\min} + \lceil (N - N_{\min}) \times (1 - S/S_{\max})^{\gamma} \rceil$$

其中 $N_{\min}$、N 分别为最小和原始预算，$S_{\max}$ 为最大可能分数，γ 控制灵敏度。当 $S \to S_{\max}$（简单编辑）时 $N_a \to N_{\min}$；当 $S \to 0$（困难编辑）时 $N_a \to N$。实验表明 $\gamma=0.15$ 为最优平衡点——NFE 稳步下降而性能保持不变。

2.2 编辑特定验证的早期剪枝

包含三个关键组件：

单步预览机制。由于编辑模型主要基于 flow matching 训练，从早期时间步 t_e 的含噪潜变量 x_{t_e} 单步估计干净潜变量：

$$x_{0|t_e} = x_{t_e} - \sigma_{t_e} \epsilon_\theta(x_{t_e}, T_{t_e})$$

其中 σ_{t_e} 为噪声尺度，ϵ_θ 为预测噪声。$x_{0|t_e}$ 解码为预览图 $I_{0|t_e}$，无需额外去噪步骤即可获得清晰预览。相比 TTS-EF 的额外去噪步骤方案，本方法在保持同等质量的同时 NFE 最低。

编辑特定验证器。在通用分数 S_{gen}（VIE-Score）基础上引入两个编辑特定指标：

(1) **编辑区域正确性** S_{reg}：通过 MLLM 识别待修改对象，Grounded SAM2 生成二值掩码 $M \in \{0,1\}^{H\times W}$。计算逐像素变化图 $\Delta = \frac{1}{C}\sum_{c=1}^{C} |I^{(c)} - I_{\text{src}}^{(c)}|$，经 softmax 归一化后在掩码内聚合：

$$S_{\text{reg}} = \sum_{H,W} M \odot \text{softmax}_{H,W}(\Delta)$$

S_{reg} 越高表示变化越集中于预期编辑区域。

(2) **指令-描述一致性** S_{cap}：MLLM 根据 I_{src} 和指令 c 生成目标描述 c_{cap}，通过 CLIP 计算一致性：$S_{\text{cap}} = \text{CLIPScore}(I, c_{\text{cap}})$。

统一评估分数组合为：$S = S_{\text{gen}} + \lambda_{\text{reg}} S_{\text{reg}} + \lambda_{\text{cap}} S_{\text{cap}}$（默认 $\lambda_{\text{reg}}=1, \lambda_{\text{cap}}=3$）。使用统一分数后，高分区间的误判从 235 降至 86（**下降 63%**），且每个编辑案例仅需一次额外 MLLM 查询。

视觉相似性过滤。使用 DINOv2 提取预览图视觉嵌入，计算两两相似度。超过阈值 $\tau_{\text{sim}}=0.9$ 的相似对中丢弃低分者，消除冗余候选。实验显示此步减少 16–24% NFE 而 G O 几乎不变。

2.3 深度优先机会性停止

剩余候选按早期分数降序排列后**逐个**深度优先生成，而非广度优先并行生成所有候选。包含两个组件：

后期保留过滤。在更晚时间步 t_l（$t_e < t_l < T$）再次生成预览并计算统一分数，自适应保留与当前最高分相当的候选，动态剪除不可能最优的样本。

实例特定验证器。通用分数常为多个候选分配相同高分，即使部分包含编辑错误。本文设计两阶段问询：(1) 给定 I_{src} 和 c，MLLM 生成 5 个与当前编辑相关的 yes/no 问题（如“肩膀是否对齐以表明正面朝向？”“头部是否正面朝前？”）；(2) MLLM 回答这些问题，计算实例特定分数 S_{spec}（yes 计数）。S_{spec} 与统一分数结合以惩罚错误候选。当 $N_{\text{high}}=4$ 个候选获得全部 yes 回答时搜索终止。最终选取最高分候选作为输出。

3. 核心实验结果

在三个 SOTA 编辑模型（Step1X-Edit、BAGEL、FLUX.1 Kontext）和三个基准（GEdit-Bench、AnyEdit-Test、Reason-Edit）上评估。效率指标包括推理效率 $\eta = \frac{1}{M}\sum_{i=1}^{M} \sigma_i \cdot \frac{S^{(i)}}{S_{\max}} \cdot \frac{NT}{\text{NFE}^{(i)}}$ 和结果效率 $\xi = \frac{1}{M}\sum_{i=1}^{M} \sigma_i \frac{\text{NFE}_{\min}^{(i)}}{\text{NFE}^{(i)}}$。

模型 / 方法	G O↑	η↑
Kontext BoN ($N=32$)	6.641	0.66
Kontext ADE-CoT	**6.695**	**1.47**
BAGEL BoN	6.908	0.69
BAGEL ADE-CoT	**6.972**	**1.27**
Step1X-Edit BoN	7.157	0.72
Step1X-Edit ADE-CoT	**7.196**	**1.45**

主要发现： • ADE-CoT 在所有模型和基准上达到与 BoN 相当或更优的性能，同时推理效率 η 提升超过 **2**×。结果效率 ξ 在 GEdit-Bench 上平均提升 **4.9**×，AnyEdit 上 **2.7**×，Reason-Edit 上 **2.9**×。• PRM 和 PARM 因通用分数在早

期阶段误判严重而性能有限。• TTS-EF 效率较高但性能下降，因为仅从早期预览中选取单个最佳样本在大规模采样时不可靠。

4. 消融实验

逐组件消融（Tab. 2，GEdit-Bench）：以 BoN 为基线（Kontext: G O=6.641, NFE=896），依次叠加：难度感知预算（NFE→797）→ 通用分数早期剪枝（NFE→719）→ 编辑特定分数剪枝（NFE→673, G O→6.647）→ 相似性过滤（NFE→508）→ 后期保留（NFE→464）→ 实例特定验证器（G O→6.702）→ 机会性停止（NFE→418）。完整流水线相比 BoN 减少 **53% NFE** 同时 G O 提升 0.054。BAGEL 和 Step1X-Edit 呈现一致趋势。

预览方式对比：额外去噪步骤（TTS-EF 方式）预览质量高但 NFE 显著增加；直接解码含噪潜变量仅在后期阶段清晰，NFE 最高；单步预览在早期即可获得清晰图像且无额外迭代，三个模型上均实现最低 NFE。

搜索策略对比：纯广度优先搜索（BFS）G O 最高但效率低；纯深度优先搜索（DFS）效率高但可能遗漏高潜力候选。ADE-CoT 的混合策略（早期 BFS + 后期 DFS）在 η 上取得最优平衡。

时间步选择：FLUX.1 Kontext 上，早期剪枝步 t_e 增至 8 时质量提升后趋于饱和；保留步 t_l 增至 16 后性能不再提升。最优设置 $t_e = 8, t_l = 16$。

MLLM 验证器影响：在 Qwen2.5-VL-72B、Qwen-VL-MAX、Qwen3-VL-32B 三种 MLLM 上评估，ADE-CoT 均实现超过 2× 加速。更强的 MLLM（Qwen3-VL）产生更高 G O 和更大效率增益（Kontext: G O=6.719, NFE=403），表明方法随验证器能力提升而受益。

机会性停止与现有方法结合：将深度优先机会性停止集成到 BoN、PRM、PARM 中，均一致减少 NFE 至少 10% 同时保持性能。$N_{\text{high}} = 4$ 为最优——性能在此饱和而 NFE 随 N_{high} 线性增长。

5. 与相关工作的对比

现有 Image-CoT 方法可分为三类：(1) **噪声缩放**（BoN [47], SANA 1.5 [88]）——通过噪声扰动生成多候选后择优，计算成本线性增长；(2) **提示干预**（Reflect-DiT [37], PARM [101]）——通过重写或反思更新增加多样性；(3) **路径搜索与剪枝**（PRM [101], PARM [101], VideoTTS [42], ICEdit/TTS-EF [103]）——利用 MLLM 评分中间状态进行剪枝。ICEdit 是首个将 Image-CoT 引入编辑的工作，但仅使用通用 VIE-Score 评估早期预览。ADE-CoT 的核心差异：(1) 难度自适应预算而非固定预算；(2) 编辑特定验证（区域定位 + 描述一致性）而非通用分数；(3) 深度优先机会性停止而非广度优先搜索。

6. 局限性与未来方向

- ADE-CoT 依赖 MLLM 验证器质量——当 MLLM 本身无法准确评估编辑正确性时，方法效果受限；
- 实例特定验证器引入额外 MLLM 查询开销，尽管通过按需调用控制成本；
- 当前仅评估单轮编辑效率，多轮编辑的级联错误累积问题需要进一步研究；
- 编辑区域正确性指标假设正确编辑主要影响掩码内像素，对全局风格变换等场景可能不适用；
- 未来方向包括：与更强 MLLM 结合以进一步提升验证精度、扩展至视频编辑的测试时缩放、以及探索强化学习指导的自适应采样策略。

CHIMERA：面向可泛化 LLM 推理的紧凑合成数据

Zhu[1], Feng[2], Sun[2], Du[2], Li[3], Saarikivi[2], Zhu[2], Meng[1] — [1]UT Austin, [2]Microsoft, [3]UGA, 2025

1. 动机与核心问题

大型语言模型（LLM）通过监督微调（SFT）和强化学习（RL）的后训练，展现出卓越的推理能力。然而，在开放和可扩展的场景中复现和扩展这些能力面临三大**数据中心挑战**：

- **冷启动问题**：缺乏包含详细长链式思维（CoT）轨迹的种子数据集来初始化推理策略。现有数据集通常仅提供最终答案或简短解释（如 GSM8K 平均解题长度仅 51.7 词），不足以训练长程复杂推理；
- **领域覆盖有限**：大多数开源推理数据集集中在数学和编码任务（如 DAPO-Math-17K、OpenR1-Math-220K、DeepMath-103K 均为纯数学），无法覆盖现实中跨学科推理需求；
- **标注瓶颈**：前沿级推理任务的难度使人工标注代价极高且不可靠——例如 Humanity's Last Exam（HLE）的专家级问题需深厚领域知识。

这些挑战对中小规模模型尤为严峻，因其更依赖高质量监督信号的结构和深度。

2. CHIMERA 数据合成流水线

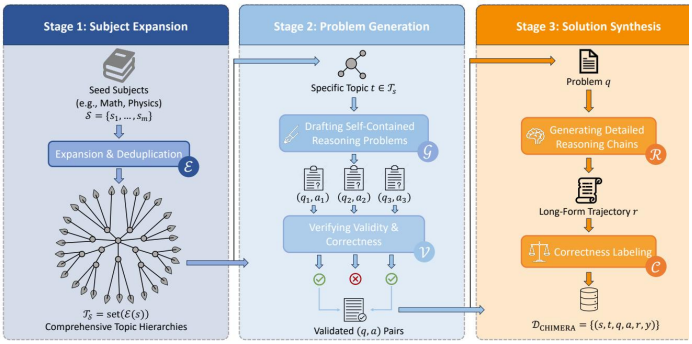


图 1：*CHIMERA* 三阶段数据合成流水线：主题扩展 → 问题生成 → 解答合成。

CHIMERA 采用模块化、全自动的 LLM 驱动合成流水线（Algorithm 1），由三个解耦阶段组成：

2.1 阶段 1：主题扩展（Subject Expansion）

从少量高层学科种子（数学、物理、化学、计算机科学等 8 个大类）出发，利用 `gpt-5` 为每个学科 s 生成尽可能全面的细粒度主题列表 $\mathcal{T}_s$。对数学因子领域过多，进行多次采样后去重。最终形成**层次化分类体系**——覆盖 8 个大学科、超过 1,179 个细粒度主题（如 "拓扑量子场论""Ramsey 理论""因果推断" 等），作为数据集的结构化骨架。该阶段与后续完全解耦，新学科可随时加入。

2.2 阶段 2：问题生成（Problem Generation）

对每个主题 t，用 `gpt-5` 生成一道问题及其答案 (q, a)。每道问题需满足三条硬约束：（1）**可解且高难度**——博士级专家可解但非开放研究问题；（2）**自包含**——所有必要信息包含在题目中；（3）**答案无歧义且可验证**——唯一确定答案，易于校验正确性。

跨模型双重验证。生成后使用两个独立 LLM（`gpt-5` 和 `o4-mini`）作为验证器 $\mathcal{V}$，分别检查问题是否良定义、答案是否正确。仅当两个模型均通过时问题才被保留，有效降低单一模型偏差和幻觉风险。

2.3 阶段 3：解答合成（Solution Synthesis）

前沿闭源模型（ChatGPT、Gemini、Claude）的详细思维过程对用户不可见，输出简洁而缺乏中间步骤。为此，CHIMERA 使用开源推理模型 **Qwen3-235B-A22B-Thinking-2507** 为每道问题 q 重新生成详尽的推理轨迹 r。随后将 r 与原始答案 a 比对，标记正确性 $y \in \{0, 1\}$：正确轨迹用于 SFT 训练，不正确的保留为 "问题 + 答案" 实例用于 RL 训练（仅需最终答案作为奖励信号）。

3. 数据集统计与特征

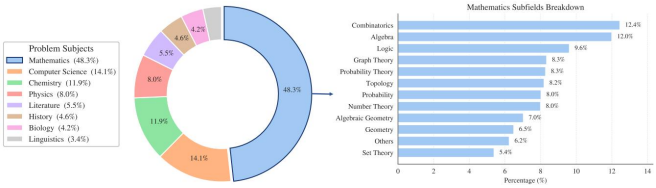


图 2：*CHIMERA* 学科分布。数学占 *48.3%*，其余涵盖 *CS*、化学、物理等 *7* 个学科。

数据集	样本数	提示词长	解答词长
GSM8K	7,473	45.1	51.7
MATH	7,500	33.0	89.5
NuminaMath-CoT	859K	44.0	205.3
OpenR1-Math-220K	225K	43.6	2,624.6
DeepMath-103K	103K	33.7	2,959.2
OpenScience	316K	76.1	1,296.8
CHIMERA	**9,225**	**211.1**	**11,121.4**

CHIMERA 仅 9,225 个样本，但问题陈述（211 词）和解答长度（**11,121 词**）远超所有现有数据集。覆盖 **8 个学科、1,179 个主题**，而现有合成数据集多为单一数学领域。数学子集进一步分解为代数拓扑、数论、组合学、概率论等数十个子领域。

4. 训练设置

基座模型：Qwen3-4B-Thinking-2507。**SFT 阶段：**在解答正确的样本上微调，batch size 256，学习率 1×10^{-5}。**RL 阶段：**从 SFT 检查点出发，使用 CISPO 算法训练 1 个 epoch，学习率 1×10^{-6}，每个提示 8 次 rollout。RL 训练集混合：(i) SFT 训练集；(ii) 合成阶段未解决但 SFT 模型在 8 次尝试内可解的问题子集。由于答案为自由形式，采用 `o4-mini` 作为奖励模型评估 rollout 质量。

评估设置：温度 0.6，top-p=0.95，top-k=20，最大 token 102,400。AIME 和 HMMT 采样 32 次，GPQA-D 采样 10 次，HLE 采样 8 次，报告无偏 pass@1。

5. 核心实验结果

模型	参数	GPQA	AIME24	AIME25	HMMT
DeepSeek-R1	671B	71.5	79.8	70.0	41.7
Qwen3-235B	235B	71.1	85.7	81.5	62.5
o4-mini (high)	–	81.4	93.4	92.7	66.7
Qwen3-4B (base)	4B	65.8	81.6	81.0	59.2
+OpenScience	4B	53.5	61.7	53.3	40.0
+CHIMERA	4B	**70.1**	**86.9**	**80.7**	**65.7**

CHIMERA 微调后 4B 模型在多项基准上取得显著提升：GPQA-D **+4.3**、AIME24 **+5.3**、HMMT Feb 25 **+6.5**、HMMT Nov 25 **+9.7**、HLE **+1.7**。**关键发现**：仅 4B 模型即可匹配或超越 8B–70B 规模基线，逼近 DeepSeek-R1（671B）和 Qwen3-235B 等两个数量级更大的模型。

相反，在 OpenScience 上微调导致全面性能下降（GPQA-D 降至 53.5，AIME24 降至 61.7）。作者推测这是因为选择题格式允许模型使用排除策略作为捷径，而非真正从头推理。

5.1 SFT vs. SFT+RL 分解

SFT 单独已贡献大部分提升（GPQA-D +3.0, AIME24 +4.9, HMMT Nov 25 +9.0, HLE +1.7），RL 提供额外增量改善（GPQA-D 再 +1.3, AIME24 再 +0.4, AIME26 +2.4）。这表明数据集本身的质量和难度是核心驱动力。

5.2 推理时扩展（Inference-Time Scaling）

pass@k 实验（$k \in \{1, 2, 4, 8\}$）显示训练后模型在所有 k 值下均持续优于基座模型。在 GPQA-D 上 pass@8 达 90.7%（基座 81.5%）；HLE 上 pass@8 从 19.5% 提升至 24.0%。**差距随 k 增大而扩大**，说明改善不仅限于单次预测，而是反映了更强的推理鲁棒性和更广的有效解轨迹覆盖。

6. 数据难度分析

在无额外微调的 Qwen3-4B-Thinking-2507 上评估各数据集：DAPO-Math-17K 达 ~88% 准确率，DeepMath-103K

和 OpenScience 同样 ~88%，OpenR1-Math-220K 约 76%——均接近饱和。而 CHIMERA 仅 **37.5%**，显示极高难度和充足的改进空间。这解释了为何先前数据集对当前模型提供的训练信号有限，而 CHIMERA 的高难度、长轨迹设计能有效驱动推理能力提升。

7. 数据质量分析

采用 LLM-as-a-Judge 盲评协议。从 HLE 随机抽取 100 道数学 +100 道物理题，从 CHIMERA 的 `gpt-5` 生成和 `gemini-3-pro` 生成各抽取同样数量。每轮将三来源各一题随机排列，由评判模型按质量排序赋分（3/2/1）。

`o4-mini` 评判结果：HLE 1.14 分，gpt-5 生成 2.67 分，gemini-3-pro 生成 2.20 分。`gemini-2.5-pro` 评判结果：HLE 1.30，gpt-5 2.53，gemini-3-pro 2.18。两个独立评判器一致表明 LLM 生成问题在**清晰度、良定义性和推理深度**上不逊于人工问题。

评估器选择经过精心设计：用最强模型生成（gpt-5、gemini-3-pro）、不同模型评判（o4-mini、gemini-2.5-pro），避免自我偏好效应。

8. 数据污染分析

使用 n-gram Jaccard 相似度检测 CHIMERA 与评估基准间的泄漏。对训练集每个问题 t_i 计算与测试集所有问题的最大 n-gram 重叠度：

$$\text{Score}_n = \frac{1}{|\mathcal{T}|} \sum_{i=1}^{|\mathcal{T}|} \max_{s \in \mathcal{S}} \frac{|G_n(t_i) \cap G_n(s)|}{|G_n(t_i) \cup G_n(s)|}$$

GPQA-Diamond 的 8-gram 和 13-gram 重叠均为**精确 0**；HLE 的 8-gram 仅 7.89×10^{-6}，13-gram 为 0。近零分数确认性能提升并非源于数据泄漏。

9. 案例研究

每条 CHIMERA 样本包含：学科标签、主题标签、形式化问题陈述、简洁可验证答案、详细推理解答（11K 词级别）和正确性标签。示例（物理/拓扑量子场论）：计算 $\mathbb{Z}_n$ 规范 TQFT 在透镜空间 $L(p,q)$ 上的配分函数 $Z(L(p,q)) = \gcd(n,p)/n$，解答从 $\pi_1(L(p,q)) = \mathbb{Z}_p$ 出发推导同态数。

10. 与相关工作的对比

现有训练数据集（GSM8K、MATH、NuminaMath 等）为人工标注且局限单一领域；合成数据集（MetaMathQA、DAPO、OpenR1 等）虽规模大但同样领域单一、解答浅短。CHIMERA 的**核心差异**在于：（1）以结构化多样性而非规模取胜——9K 样本覆盖 8 个学科；（2）解答长度达 11K 词，远超同类 4–10 倍；（3）全自动质量控制消除人工依赖；（4）跨模型双重验证确保数据可靠性。

11. 局限与展望

- 合成数据质量受限于生成模型能力（gpt-5 和 Qwen3-235B），可能继承其偏差和盲区；
- 当前仅验证了 4B 模型，在更大/更小规模模型上的效果有待探索；
- 评估基准以 STEM 为主，更广泛的人文社科推理覆盖不足；
- 数据集规模紧凑（9K），与大规模数据结合可能带来进一步收益；
- LLM-as-a-Judge 评估协议本身存在评判偏差风险；
- 未来方向：扩展至多模态推理、探索数据规模-质量-难度的最优平衡、结合持续学习适应新领域。

核心结论：结构化、高质量的合成数据——而非单纯的规模——在赋能 LLM 推理能力中发挥核心作用。CHIMERA 以仅 9K 样本使 4B 模型达到 671B 模型的竞争水平，为数据高效的推理后训练提供了有力证据。

VGGT-Det：挖掘 VGGT 内部先验实现无传感器几何的多视角室内 3D 目标检测

Cao[1], Wu[1,3], Chen[2], Zhong[1], Hong[2], Xu[1] — [1]HKUST, [2]Huawei, [3]SYSU, 2025

1. 问题动机与背景

多视角室内 3D 目标检测在机器人和增强现实中具有广泛应用。现有方法（ImVoxelNet、NeRF-Det、MVSDet 等）严重依赖**传感器提供的几何输入**——精确标定的多视角相机位姿或深度图。然而，这些几何信息在实际场景中获取成本高昂且往往不可用（室内相机通常手持或频繁移动），严重限制了系统的可部署性。

本文提出一个更实际的设置：**无传感器几何（Sensor-Geometry-Free, SG-Free）**多视角室内 3D 目标检测——不使用任何传感器提供的几何输入（多视角位姿或深度）。该设置极具挑战性，因为模型需要仅从无位姿的 2D 图像中推断 3D 结构并完成检测。

近期，Visual Geometry Grounded Transformer（VGGT）等前馈式 3D 重建模型表明，强 3D 线索可以直接从图像中推断。VGGT-Det 正是基于这一洞察，**首次**针对 SG-Free 设置设计的 transformer 检测框架。其核心思想不是简单消费 VGGT 的预测输出，而是深入挖掘 VGGT 编码器**内部**的语义和几何先验，通过两个关键组件——Attention-Guided Query Generation (AG) 和 Query-Driven Feature Aggregation (QD)——有效释放 VGGT 的潜力。

2. 系统架构概览

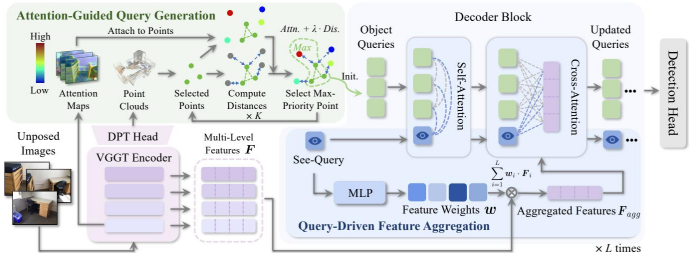


图 1：*VGGT-Det* 整体框架。*VGGT* 编码器提取 *3D* 感知特征，解码器通过 *AG* 生成的 *object query* 与 *QD* 聚合的多层级几何特征交互，迭代更新 *query* 实现检测。

VGGT-Det 采用**编码器-解码器 transformer** 架构。给定多视角图像 $\{I_1, I_2, \ldots, I_V\}$，冻结的预训练 VGGT 编码器提取 3D 感知 token。各视角 token 拼接为统一表示：

$$\mathbf{T}_{\text{concat}} = [\mathbf{T}_1; \mathbf{T}_2; \ldots; \mathbf{T}_V] \in \mathbb{R}^{(V \cdot M) \times C}.$$

基础骨干网络借鉴 3DETR，通过最远点采样（FPS）从 VGGT 预测的点云 $\mathbf{P}_{\text{pred}}$ 中采样 K 个点，编码为初始 object query $\mathbf{Q}_0$。query 经 L 层 decoder 迭代：每层先自注意力交互，再与 $\mathbf{T}_{\text{concat}}$ 交叉注意力，最终通过检测头输出类别 $\hat{\mathbf{c}} \in \mathbb{R}^K$ 和边界框 $\hat{\mathbf{b}} \in \mathbb{R}^{K \times 7}$。

关键问题：VGGT 预测的点云是场景的稠密重建，不区分前景/背景。FPS 均匀采样导致大量 query 落在背景区域，严重损害检测效果。同时，简单使用 VGGT 最后一层特征忽略了其逐层从 2D 到 3D 的渐进几何编码。AG 和 QD 分别针对这两个问题设计。

3. Attention-Guided Query Generation (AG)

3.1 核心观察：VGGT 注意力蕴含语义信息

尽管 VGGT 未在语义任务上训练，作者发现其编码器注意力图与语义内容高度相关——物体区域倾向于获得更高的注意力权重。这一有趣观察激发了 AG 的设计。

3.2 算法细节

设 VGGT 重建点云 $\mathbf{P} \in \mathbb{R}^{N \times 3}$，注意力权重 $\mathbf{A} \in \mathbb{R}^N$。目标是采样 K 个 query 点，同时兼顾语义重要性和空间多样性。

步骤 1：归一化注意力权重：

$$\mathbf{A}_{\text{norm}} = \frac{\mathbf{A} - \min(\mathbf{A})}{\max(\mathbf{A}) - \min(\mathbf{A}) + \epsilon}.$$

步骤 2：选择注意力最高点作为第一个 query：$\mathbf{I}[1] = \arg\max \mathbf{A}_{\text{norm}}$。

步骤 3：后续点通过融合优先级迭代选择：

$$\text{Priority} = \mathbf{A}_{\text{norm}} + \lambda_{\text{dist}} \cdot \mathbf{D}_{\text{norm}},$$

其中 $\lambda_{\text{dist}} \in [0, 1]$ 为权衡系数（最优值 0.8），$\mathbf{D}_{\text{norm}}$ 为到已选点集的归一化最小欧氏距离：

$$\mathbf{D}_{\text{min}} = \min_{j \in \{1, \ldots, k-1\}} \|\mathbf{P} - \mathbf{P}_{\mathbf{I}[j]}\|_2.$$

每步选取 $\mathbf{I}[k] = \arg\max_{i \notin \mathcal{S}} \text{Priority}$。该设计鼓励 query 集中于语义物体区域，同时保持空间覆盖的多样性。

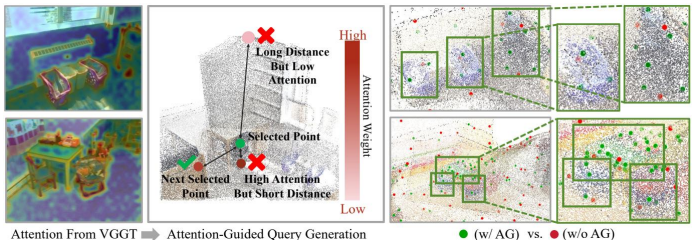


图 2：*AG* 可视化。左列：*VGGT* 注意力图突出物体区域；中列：*AG* 采样过程；右列：相比 *FPS*（红点），*AG* 采样（绿点）更集中于物体区域（绿框）。

4. Query-Driven Feature Aggregation (QD)

VGGT 编码器跨层**渐进地**将 2D 特征提升为 3D 表示，每层编码不同层级的几何抽象。QD 引入可学习的 **See-Query** $\mathbf{q}_{\text{see}} \in \mathbb{R}^C$，与 object query 交互后动态聚合多层级特征。

4.1 多层级特征聚合

设 VGGT 产生 L 个中间层特征 $\{\mathbf{F}_1, \ldots, \mathbf{F}_L\}$，$\mathbf{F}_i \in \mathbb{R}^{N \times C}$。See-Query 通过 MLP+Softmax 生成层级权重：

$$\mathbf{w} = \text{Softmax}(\text{MLP}(\mathbf{q}_{\text{see}})), \quad \sum_{i=1}^{L} \mathbf{w}_i = 1.$$

聚合特征：$\mathbf{F}_{\text{agg}} = \sum_{i=1}^{L} \mathbf{w}_i \cdot \mathbf{F}_i$，作为解码器交叉注意力的 key/value。

4.2 See-Query 与 Object Query 的交互

See-Query 与 object query 拼接为统一集合：

$$\mathbf{Q}_{\text{input}} = [\mathbf{q}_{\text{see}}, \mathbf{q}_1, \ldots, \mathbf{q}_K] \in \mathbb{R}^{(K+1) \times C}.$$

经自注意力后，See-Query“看到”object query 的需求，再通过交叉注意力与 $\mathbf{F}_{\text{agg}}$ 交互：

$$\mathbf{Q}_{\text{cross}} = \text{CrossAttention}(\mathbf{Q}_{\text{self}}, \mathbf{F}_{\text{agg}}).$$

更新后的 See-Query 传入下一层 decoder，迭代精炼实现动态、上下文感知的多层级几何特征聚合。

5. 实验结果

5.1 数据集与设置

在 **ScanNet**（18 类）和 **ARKitScenes**（17 类）上评估，报告 mAP@0.25。VGGT 编码器训练时冻结，使用 256 个 object query，AdamW 优化器（lr 2.5×10^{-4}，weight decay 10^{-4}），cosine 退火调度，8×H800 GPU 训练约 2 天。

5.2 ScanNet 主实验

方法	mAP@0.25
ImVoxelNet	35.2
FCAF3D	40.6
NeRF-Det	41.2
MVSDet	42.5
VGGT-Det (Basic backbone)	41.4
VGGT-Det (BB+AG)	44.2
VGGT-Det (BB+AG+QD)	**46.9 (+4.4)**

为公平比较，所有竞争方法均在 SG-Free 设置下重训练——使用 VGGT 预测的位姿/点云替代传感器几何。VGGT-Det 以 **46.9 mAP@0.25** 显著超越 MVSDet（42.5）**4.4** 个点，超越 FCAF3D（40.6）**6.3** 个点。

5.3 ARKitScenes 主实验

在 ARKitScenes 上，VGGT-Det 达到 **28.0 mAP@0.25**，超越 MVSDet（19.4）**8.6** 个点。在 refrigerator 类别上优势尤其显著（57.7 vs 25.5）。部分类别（如 TV、stove）因物体小薄或嵌入墙体，所有方法均表现不佳。

5.4 消融实验

AG 的贡献： AG 在基础骨干上提升 **+2.8** mAP，验证了 VGGT 注意力语义先验的有效利用。验证损失动态分析显示，加入 AG 后 GIoU 损失显著低于 baseline，证实注意力引导改善了定位精度。

QD 的贡献： 在 AG 基础上，QD 再提升 **+2.7** mAP。与仅用最后一层特征（Vanilla-v1, 37.5 mAP）和固定四层顺序聚合（Vanilla-v2, 44.2 mAP）相比，QD 的自适应聚合达到 46.9 mAP。训练初期 AG 和 AG+QD 损失相近（See-Query 仍在学习），数个 epoch 后 AG+QD 损失显著降低，证实 See-Query 学会了有效聚合几何特征。

λ_{dist} 敏感性： $\lambda_{\text{dist}} = 0.8$ 最优（46.9），0.9 降至 44.9，0.5 降至 40.7，表明语义引导和空间分散需要恰当平衡。

输入帧数： 20 帧 42.6、40 帧 45.3、60 帧 46.2、80 帧 46.9、100 帧 47.3，80 帧左右性能趋于饱和。在所有帧数下均一致超越 MVSDet。

5.5 噪声鲁棒性

向 VGGT 点云添加高斯噪声（$\sigma = \text{range} \cdot \text{noise_level}$）。在 noise_level=0.01 时，VGGT-Det 保持 47.0 mAP，而 FCAF3D 降至 18.7（差距 **28.3** 个点）。noise_level=0.3 时 VGGT-Det 仍有 34.1 mAP，FCAF3D 已降至 0.0。鲁棒性源于 AG 减少了对点云精确几何的依赖，转而利用 VGGT 内部语义先验。

5.6 效率分析

组件	时间 (s)	显存 (GB)	mAP
VGGT	0.68	11.71	–
+MVSDet	0.21	13.81	39.7
+Ours	0.23	**3.57**	**45.3**

40 帧设置下，VGGT-Det 与 MVSDet 推理时间相当（0.23s vs 0.21s），但显存仅 **3.57 GB**（vs 13.81 GB），减少约 74%，同时 mAP 提升 5.6 个点。

5.7 SG-Free 与 SG-Based 的差距

使用传感器几何时（+SG），VGGT-Det 达到 **58.8** mAP，同样为最高。SG-Free（46.9）与 SG-Based 之间仍有约 12 个点差距，符合预期——传感器几何提供了强先验。但考虑到传感器几何在实际场景中的不可用性，SG-Free 设置具有重要实际价值。

6. 与相关工作的对比

室外 vs 室内： 室外方法（DETR3D、BEVFormer 等）假设相机刚性安装于车辆，位姿易得；室内相机手持/移动，位姿获取困难，凸显 SG-Free 的必要性。

已有室内方法： ImVoxelNet 通过反投影聚合 2D 特征至 3D 体素；NeRF-Det 集成 NeRF 几何约束；MVSDet 用平面扫描 + 概率采样正则化深度。这些方法均**依赖传感器位姿**。

3D 重建模型： DUSt3R/MASt3R 受限于成对输入，需全局对齐；Fast3R 支持长序列单次前向；VGGT 额外预测相机位姿和 3D 属性。VGGT-Det 的独特之处在于**深入挖掘 VGGT 内部先验**而非仅用其输出。

7. 局限性与未来方向

- **VGGT 开销：** VGGT 推理带来显著运行时和显存开销（0.68s, 11.71GB），在所有 SG-Free 流水线中是瓶颈；
- **尺度归一化：** VGGT 产生归一化预测，当前需利用数据集尺度信息进行反归一化，限制了完全的传感器无关性；
- **小/薄物体：** 无精确传感器几何时，小型、薄型或嵌入式物体（如 TV、stove）检测仍普遍困难；
- **未来方向：** 引入更轻量的 VGGT 式模型并支持度量尺度预测；扩展至更多室内场景和更大规模数据集。

RubricBench：对齐模型生成评分标准与人类标准

Zhang*, Zhou*, Wang, Lyu, Ming, Xu, Sun, Zheng, Peng, Liu, Ma† — CityU HK, Tencent, MBZUAI, McGill-Mila 等, 2025

1. 动机与问题定义

随着 LLM 输出从简单补全进化为复杂推理密集型生成，奖励模型（RM）面临根本瓶颈：倾向于优先考虑**表面复杂度**而非用户意图的实际满足。标量 RM 将偏好压缩为不透明分数，易受 reward hacking；生成式 RM（LLM-as-a-Judge）通过 CoT 推理提升可解释性，但缺乏显式约束时仍易产生事后合理化。因此，领域正转向 **Rubric-guided 评估**——将模糊的质量定义分解为原子化、可验证的约束条件。

然而，社区缺乏统一基准来评估这一范式。现有基准存在三大缺陷：

- 依赖饱和或过时样本，缺乏区分现代高性能模型的复杂度；
- Rubric 方法常在分散的自定义数据集上评估，阻碍跨方法比较；
- **最关键**：缺乏人类级别的 rubric 标注——无法度量模型生成 rubric 与理想评估标准的差距。

RubricBench 包含 **1,147 个成对比较**，每个样本配有专家标注的原子化 rubric，专为评估 rubric 引导评估的可靠性而设计。

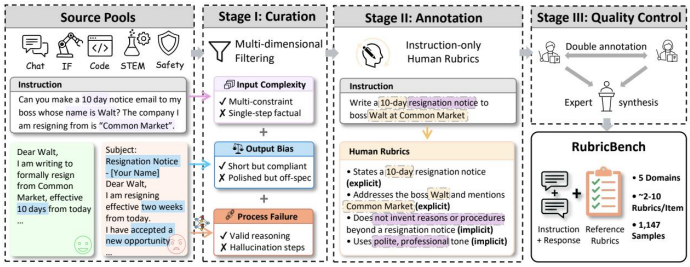


图 1：*RubricBench* 构建流程与评估设置。从已有偏好数据出发，经多维过滤筛选挑战性样本，通过三阶段流水线标注 *instruction-only* 的人类 *rubric*。

2. 基准构建

2.1 设计原则

三大原则：(1) **判别性难度**——优先选择表面线索（冗长、格式）与实际质量矛盾的样本；(2) **指令派生**——rubric 仅从 instruction 推导，不接触候选回复，防止 response-aware 泄漏；(3) **原子验证**——rubric 表述为独立的二值（Yes/No）约束，支持粒度诊断。

2.2 数据来源与领域覆盖

样本来自 HelpSteer3、PPE、RewardBench2 等高质量基准，覆盖 Chat（36.5%）、Coding（23.9%）、STEM（23.8%）、Instruction Following（8.8%）和 Safety（7.0%）五个领域。大多数样本含 4–6 个 rubric 项，标注粒度跨领域一致。

2.3 阶段 I：多维数据过滤

沿三个独立维度审查候选样本，不满足任一条件者过滤：

输入复杂度。优先选择复合指令，区分显式约束（如" 列出三个理由""避免循环"）和隐式约束（如向祖父母解释" 区块链" 隐含需避免术语）。

输出表面偏差。保留被拒回复充当表面干扰器的样本对：(1) 长度偏差：被拒回复 $\geq 1.5\times$ 于优选；(2) 格式偏差：被拒回复具有更优结构（JSON/Markdown/LaTeX）；(3) 语调偏差：被拒回复表现出更高置信度或专业术语。

过程失败。优先推理依赖型样本——正确结论可能掩盖有缺陷的中间步骤。使用多个 judge 模型生成评估 CoT，仅保留含 2+ 种推理谬误的样本：幻觉步骤、逻辑不一致、指令约束在推理过程中侵蚀。

2.4 阶段 II：Rubric 标注协议

标注者为每条指令开发" 可执行规范"，遵循两项标准：(1) **结构原子性**：每个 rubric 含 2–10 项，每项表述为单一二值检查，不含内部冲突；(2) **语义客观性**：标注者不知候选回复，rubric 仅从指令推导，映射至 Reasoning/Content/Expression/Alignment/Safety 域。包含显式约束和隐式需求（如" 为老年人步行游" 隐含需休息点和无障碍路线）。

2.5 阶段 III：质量控制

三阶段协议：(1) **专家协调**——双独立标注后由资深审核者合成统一 rubric，仅保留共识标准；(2) **结构验证**——检查逻辑一致性、最小冗余、指令对齐；(3) **压力测试**——对安全和推理任务抽查，在留出模型回复上验证判别力。

3. 实验设置

3.1 评估条件

为隔离 rubric 质量影响，在三种控制条件下评估（backbone、prompt、解码参数固定）：(1) **Vanilla**——直接生成偏好裁决，无显式中间推理；(2) **Self-Generated Rubrics**——RM/judge 先从指令推导 rubric 再验证；(3) **Human-Annotated Rubrics**——注入人类 rubric，绕过 rubric 瓶颈，作为上界。

3.2 模型覆盖

四类范式：(1) 标量 RM（ArmoRM, InternLM2-Reward, Tulu-3-RM）；(2) 生成式 RM（Nemotron-GenRM-49B, Nemotron-BRRM-14B, RM-R1-32B）；(3) LLM-as-a-Judge（GPT-4o-mini, DeepSeek-v3.2, Gemini-3-Flash 等）；(4) Rubric-Aware Judges（Auto-Rubric, RocketEval, CheckEval, TICK, OpenRubric）。

3.3 评估指标

偏好准确率：$\text{Acc} = \frac{1}{|\mathcal{D}|}\sum_{i\in\mathcal{D}} \mathbb{I}[\hat{z}_i = z_i^\star]$。

Rubric 对齐指标：设生成 rubric $\tilde{\mathcal{R}} = \{\tilde{r}_1, \ldots, \tilde{r}_K\}$，参考 rubric $\mathcal{R} = \{r_1, \ldots, r_M\}$。Rubric Recall $= H/M$（参考项被匹配比例）；Hallucination Rate $= \frac{1}{K}\sum_k \mathbb{I}[\text{无匹配}]$；Structural F1 为 Recall 与 Precision（1−Hallucination Rate）的调和均值。

4. 核心实验结果

4.1 主结果与性能层次

模型/方法	Overall Acc
标量/生成 RM（ArmoRM 等）	44–51%
LLM-as-a-Judge（GPT-4o-mini）	40.2%
FARE（微调 Judge）	54.5%
Self-Gen Rubric 最优（OpenRubric+Gemini）	58.1%
Human Rubric（OpenRubric+DeepSeek）	**84.9%**
Human Rubric（TICK+Gemini）	83.0%

隐式推理不足：标量和生成 RM 难以稳定超越随机（44–50%），标准 LLM judge 同样表现不佳。**Rubric-aware 流水线恢复性能：**自生成 rubric 一致性提升 vanilla 基线（GPT-4o-mini +6%，DeepSeek +19%），最强配置达高 50s。**注入人类 rubric 产生最大跃升：**准确率飙升至 ∼85%，因 backbone 和验证流程不变，+27% 的增量有效隔离了 rubric 错误规范化为当前自动评估的主导失败模式。

失败集中于 Safety 域：自生成方法在安全边界执行上失败（Acc ≈ 25–30%），人类 rubric（显式编码拒绝逻辑）将性能恢复至 >90%。**执行天花板：**即使人类 rubric，准确率在 ∼85% 而非 100% 处饱和，反映开放偏好中的不可约歧义和执行错误。

4.2 Rubric Gap 量化

跨所有模型家族（GPT-4o-mini 到 Gemini-3-Pro、GPT-OSS-120B、Qwen3.5-Plus），人类 rubric 相比自生成 rubric 的**稳定增益约 ∼27%**（Δ 范围 22.1–28.3%）。这一差距在最新前沿推理模型上依然持续，表明主要限制不是推理能力而是 **rubric 形成能力**。

5. 测试时计算无法弥合差距

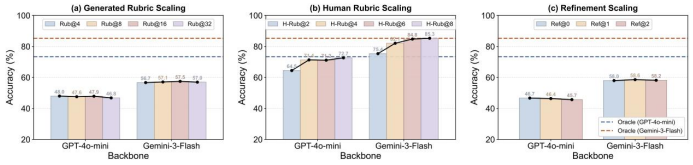


图 2: 测试时缩放结果。*(a)* 缩放自动 *rubric* 数量；*(b)* 缩放人类 *rubric*；*(c)* 缩放迭代精炼深度。

(a) 增加合成 rubric 采样数：收益递减且非单调——GPT-4o-mini 从 Rub@4 的 48.0% 退化至 Rub@32 的 46.8%，Gemini-3-Flash 在 56.7–57.5% 附近平坦，额外样本主要积累噪声。**(b) 缩放人类 rubric（随机子采样）：**鲁棒正相关——Gemini 从 H-Rub@2 的 75.4% 提升至 H-Rub@8 的 85.3%，GPT-4o-mini 从 64.5% 至 72.7%。**(c) 迭代精炼深度：**同样无法弥合差距，甚至轻微下降。结论：**瓶颈在 rubric 质量而非计算量**。

6. 深入分析

6.1 认知错位：结构质量分析

方法	Avg.#	Recall	Halluc.	F1
Auto-Rubric	13.2	40.4%	76.2%	28.3
RocketEval	4.4	35.4%	54.1%	38.3
CheckEval	14.6	53.8%	68.7%	38.2
TICK	6.3	26.3%	74.1%	24.8
OpenRubric	15.4	47.5%	72.6%	31.5

核心发现——**注意力置换**（Attention Displacement）：模型将生成预算浪费在无关 rubric 上。尽管 Auto-Rubric 和 OpenRubric 平均生成 >13 项，幻觉率仍 >70%，同时遗漏近半关键约束。减少噪声的方法（如 RocketEval 仅 4.4 项）以牺牲覆盖率为代价。CheckEval 达最高 Recall（53.8%），可能因其依赖人工策展的高层标准来种子生成。

6.2 Rubric 特征诊断

沿**约束刚性**（R）和**意图必要性**（N）两个正交维度分析原子标准。LLM 生成的 rubric 含显著更多低必要性规则（N=1：17.9% vs 10.1%）和极端刚性规则（R=5：12.8% vs 7.7%），产生更多高刚性/低必要性标准（13.7% vs 8.4%）。刚性与必要性的耦合远弱于人类（$\mathrm{corr}(\mathrm{R},\mathrm{N}) = 0.133$ vs 0.306），表明模型常生成过度严格但非必要、或必要但规范不足的规则。

6.3 价值反转案例

案例 1（不可能任务）：指令要求”将 SQL 转 Mongo 处理所有情况”。人类 rubric 编码可行性约束（需承认”所有情况”不可能），模型 rubric 退化为实现清单（检查特定库）。结果：模型惩罚正确拒绝的回复 B（”缺少代码”），奖励幻觉方案回复 A。

案例 2（欠指定任务）：指令缺少利率信息。人类 rubric 要求认识论谦逊（请求缺失信息），模型 rubric 注入假设并验证基于任意假设的数学，奖励编造数据的回复。

6.4 执行失败模式

即使提供人类 rubric，模型 judge 仍展现四类系统性执行错误：(1) **软约束谬误**——识别 rubric 项但在最终决策中不作为硬约束执行；(2) **隐式重加权**——以不同于人类意图的方式重排 rubric 标准优先级；(3) **缺失决策语义**——rubric 不判别时引入额外轴而非承认平局；(4) **抗拒拒绝**——任务不可行时 pivot 至可编译性等次要标准，未能操作化 rubric 隐含的拒绝行为。

7. 人类验证

在 100 个随机实例上，人类评估者使用人类 rubric 达 **92.0%** 准确率，使用模型生成 rubric 降至 **61.0%**；Gemini 分别为 83.0% 和 54.0%。这证实瓶颈在 rubric 质量而非评估者推理能力。Rubric 匹配的 inter-annotator agreement：模型间 0.85，人机间 0.79，确认指标计算基础稳健。

8. 讨论与局限

三大结论：(1) RubricBench 有效区分 RM 能力——从标量 RM 的 ∼44% 到 rubric-aware 的 ∼58% 再到人类 rubric 的 ∼85%，形成清晰层次；(2) 量化了严重的 ∼27% rubric gap，且该差距对模型规模免疫——人类 rubric 随规模一致有效，模型 rubric 严重收益递减；(3) 根因是认知错位——模型擅长检查显式指令但无法自主定义必要约束。

未来方向：对齐 rubric 内容是关键——需开发使模型内化人类优先级层次的方法；更结构化的 rubric 设计（区分硬/软约束、显式权重）有助弥合执行差距。

局限性：(1) 数据从公开基准再策展，分布受源数据集范围约束，可能不完全覆盖专有领域长尾场景；(2) 依赖高质量专家标注限制了规模（相比纯合成数据集）；(3) 严格二值清单形式优先可验证性而非细微差别，可能不完美捕捉高度主观任务（如创意写作）中的连续质量。

OpenAutoNLU：面向自然语言理解的开源 AutoML 库

Arshinov, Boriskin*, Senichev*, Zaripov, Galimzianova, Karpov†, Sanochkin† — MWS AI / ITMO / MBZUAI, 2025

1. 动机与问题定义

文本分类与命名实体识别（NER）是 NLP 基础任务。实际部署面临方法选择困难、数据质量问题、OOD 检测需求及配置复杂等挑战。现有框架（H2O、LightAutoML）主要面向表格数据；AutoGluon 缺乏 OOD 检测与数据质量评估；AutoIntent 不支持 NER 且 OOD 检测需训练样本。

OpenAutoNLU 是专为 NLP 设计的开源 AutoML 库，核心创新在于**数据感知的训练方案自动选择**，同时集成数据质量诊断、无监督 OOD 检测和 LLM 增强功能。

2. 系统架构

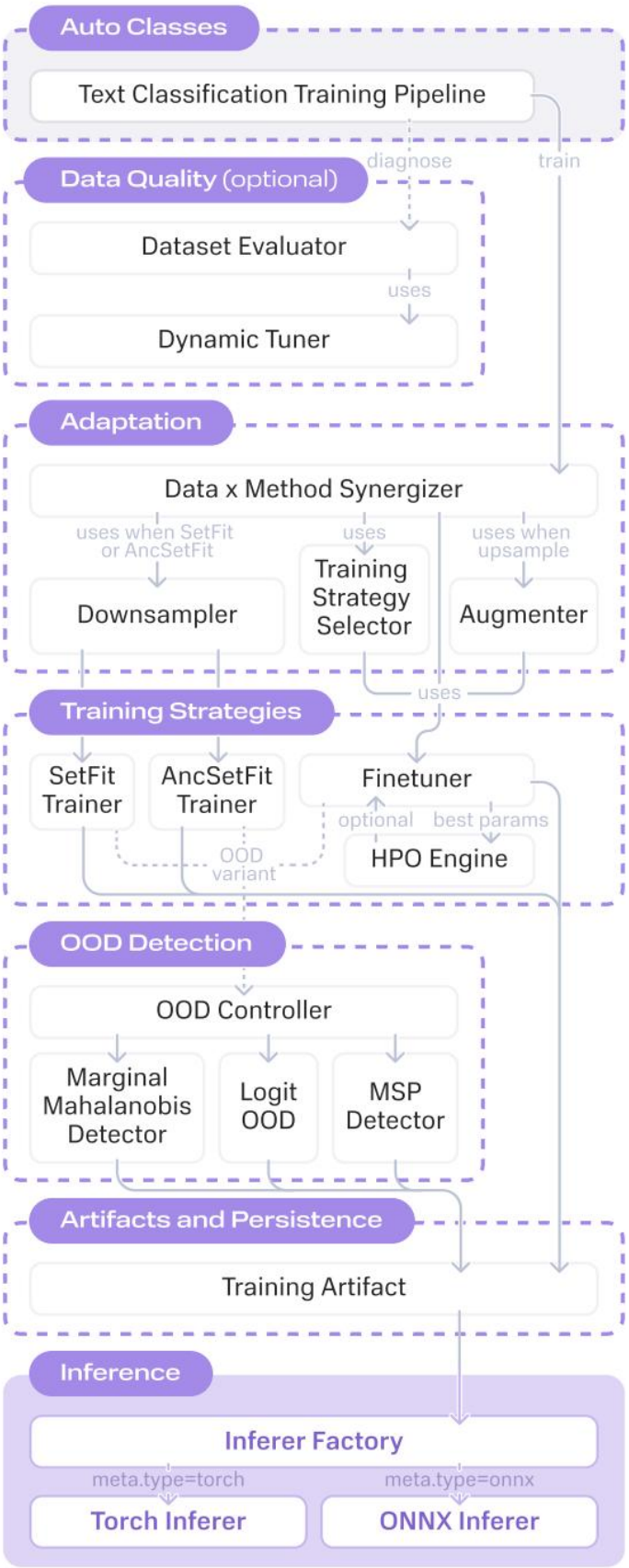


图 1：*OpenAutoNLU* 文本分类训练与推理流水线。

系统由 `auto_classes` 模块暴露四个公共流水线类，管理数据加载、质量评估、方法选择、训练、评估和模型导出的完整生命周期。

2.1 数据感知训练方案选择（核心创新）

系统根据每类最小样本数 $n_{\min}$ 自动选择训练方案：$2 \leq n_{\min} \leq 5$ 使用 **AncSetFit**（锚点对比学习）；$5 < n_{\min} \leq 80$ 使用 **SetFit**（句子 Transformer + 逻辑回归头）；$n_{\min} > 80$ 使用**全量微调**（配合 Optuna HPO）。阈值通过公开英文及内部非英文数据集的广泛实验确定。

2.2 数据级优化

方法选择前执行自适应重平衡：若低资源类别（$n \leq 80$）占比超阈值（默认 0.3），则上采样至 $n = 81$（Augmentex 扰动或 LLM 释义），提升至微调方案；反之，过多表示的类别被下采样以保持训练平衡。

2.3 训练方法模块

包含 Finetuner（标准微调 + early stopping + 可选 HPO）、SetFitMethod（对比句子 Transformer）、AncSetFitMethod（锚点增强版）及 TokenClassificationFinetuner（BIO 标注 NER），每种均有对应 OOD 变体。

2.4 数据质量诊断

DynamicTuner 记录逐样本 logit，四种评估器消费这些信号：**Retag**（标注错误）、**Uncertainty**（低置信样本）、**V-Information**（低信号样本）、**Dataset Cartography**（easy/ambiguous/hard 区域划分）。NER 使用基于 Dawid–Skene 的 Monte Carlo dropout 共识估计。

2.5 OOD 检测

微调使用边际 Mahalanobis 距离，SetFit 使用 MSP，AncSetFit 使用 outOfScope 类。合成 OOD 样本由 gibberish 生成器产生，阈值在验证集上调优。关键优势：**无需 OOD 训练样本**。

2.6 LLM 集成与 NER

LLM 模块支持数据增强、领域分析和合成测试集生成（每类 ≤ 80 样本时与真实测试集差异 < 5 pp）。NER 流水线支持 offset/bracket 标注格式，实体级评估与部分匹配。所有方法支持 ONNX 导出，推理管理器自动检测硬件并防止 OOM。

3. 实验评估

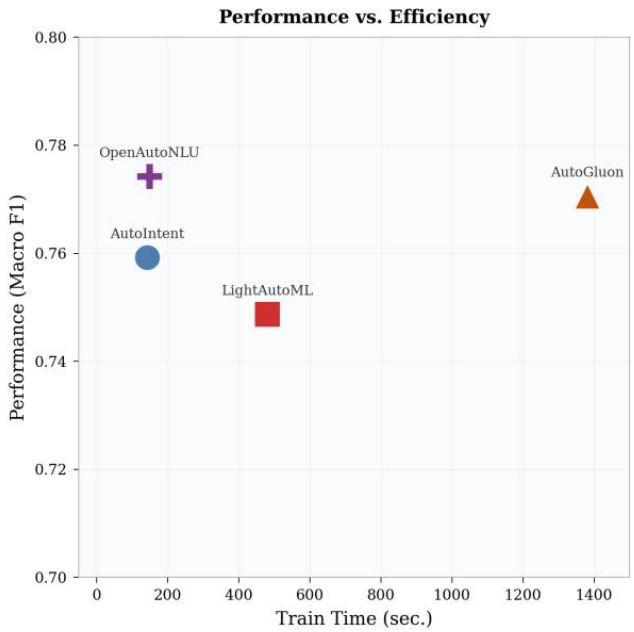


图 2：各框架 *macro-F1* 与训练时间的性价比（四数据集平均）。

在 Banking77、HWU64、MASSIVE、SNIPS 上评估，覆盖 low-/medium-/full-data 方案，每配置 3 个随机种子取平均。

3.1 In-domain 分类

框架	Bank77	HWU64	MASSIVE	SNIPS
OpenAutoNLU	0.912	**0.890**	**0.876**	**0.921**
AutoGluon	**0.920**	0.902	0.861	0.763
AutoIntent	0.869	0.829	0.755	0.786
LightAutoML	0.907	0.900	0.841	0.761
H2O	0.645	0.642	0.701	0.778

OpenAutoNLU 在 4 个数据集中 3 个最优。AutoGluon 仅在 Banking77 略优但计算成本显著更高。

3.2 跨数据规模表现

框架	5–10 shot	40–80 shot	Full
OpenAutoNLU	0.687	0.809	0.915
AutoIntent	0.766	0.814	0.749
AutoGluon	0.607	0.876	0.915
LightAutoML	0.541	0.868	0.905
H2O	0.345	0.668	0.712

OpenAutoNLU 凭借自适应方案选择在**所有数据规模上均保持稳健**，是唯一全范围表现一致的框架。

3.3 OOD 检测评估

框架	模式	Macro F1	OOD F1
Banking77			
OpenAutoNLU	unsup	**0.912**	**0.433**
AutoIntent	CL+sup	0.774	0.156
MASSIVE			
OpenAutoNLU	unsup	**0.876**	**0.515**
AutoIntent	CL+sup	0.683	0.366

OpenAutoNLU 无监督模式在 Macro F1 和 OOD F1 上均领先，且无需 OOD 训练样本。AutoIntent 即使有监督也大幅落后。

4. 与现有框架的系统对比

- **vs. AutoIntent**：full-data 设定全面领先（MASSIVE 差距达 12.1 pp）；OOD 检测无需训练样本；额外支持 NER；
- **vs. AutoGluon**：仅 Banking77 略逊 0.8 pp，训练时间显著更短；不支持 OOD 和数据质量诊断；
- **vs. LightAutoML/H2O**：表格 AutoML，文本仅 Word2Vec，NLP 性能全面落后；
- **独有功能**：数据质量评估、无监督 OOD、LLM 增强、ONNX 推理。

5. 局限性与未来方向

当前基于 $n_{\min}$ 阈值的确定性选择不保证对所有数据集最优，未来计划实现元模型自动决定最优组合。阈值对更多语言的泛化性待验证。LLM 生成测试集在大数据规模时质量下降。NER 尚无少样本方案自动选择。面向受限硬件的适配未探索。

CoVe：基于约束引导验证的交互式工具使用智能体训练

Chen*, Gong*, Li*, Liu, Tian, Fu, Wu, Zhang, Zhang, Zhang, Tu✉, Liu — Huawei Research, 2025

1. 动机与问题背景

多轮交互式工具使用智能体（interactive tool-use agent）是 LLM 应用的核心方向，但面临根本性挑战：现实用户需求**复杂、隐含且模糊**，而底层工具（API）严格要求**确定性动作和精确参数**。智能体不仅需理解模糊指令，还须通过多轮对话澄清用户意图，再转化为可执行命令。

训练此类智能体的关键瓶颈在于**高质量数据获取**：人工标注成本高昂且难以规模化；现有合成方法依赖 LLM 生成用户-智能体交互轨迹并由 LLM 验证正确性，但存在三个根本缺陷：

- LLM 固有的不可控性无法保证查询的**可解性**和验证的**绝对正确性**；
- LLM 能力受限，生成任务的工具调用次数和对话轮数有限，**缺乏复杂样本**；
- LLM 评估存在幻觉风险，验证结果不可靠。

CoVe（Constraint-Verification）应运而生：一个后训练数据合成框架，通过**显式任务约束**同时保证数据复杂度和轨迹正确性。约束扮演双重角色——引导生成复杂轨迹，并作为确定性验证器评估轨迹质量。

2. 问题形式化

将多轮交互建模为简化 POMDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T})$：

- $\mathcal{S}$：隐状态空间，包含用户隐含意图 u 和外部环境状态 e；
- $\mathcal{A} = \mathcal{A}_{\text{tool}} \cup \mathcal{A}_{\text{resp}}$：动作空间，智能体可执行结构化工具调用 $a \in \mathcal{A}_{\text{tool}}$ 或生成自然语言回复 $a \in \mathcal{A}_{\text{resp}}$；
- $\mathcal{O}$：观察空间，每步 t 的观察 o_t 为用户指令 o^{user} 或工具执行结果 o^{env}；
- $\mathcal{T}(s_{t+1} \mid s_t, a_t)$：状态转移函数。

交互产生轨迹 $\tau = [o_1, a_1, \ldots, o_T, a_T]$，目标是生成满足用户隐含意图 u 且遵守环境约束 e 的轨迹。

3. CoVe 框架

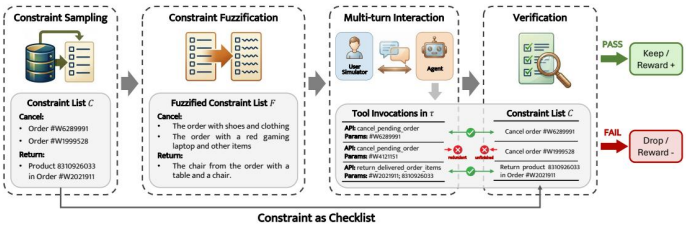


图 1：*CoVe* 框架概览。显式约束经模糊化后引导用户模拟器生成多轮交互，原始约束作为确定性清单验证轨迹。

3.1 约束采样（Constraint Sampling）

从沙盒数据库和可用工具列表定义的确定性预任务环境状态中，随机采样约束集 $C = \{c_1, c_2, \ldots, c_n\}$。例如在 Retail 域中，$c_i$ 可指定取消某订单或退回某商品。由于 C 直接源于已有数据库记录，任务**可解性**在逻辑上得到保证。采样阶段使用精确标识符（Order ID、Product ID）引用相关元素。

3.2 约束模糊化（Constraint Fuzzification）

直接暴露精确 ID 会将问题退化为简单的槽填充。现实用户很少记得精确标识符，而是通过组成、属性或上下文描述对象。CoVe 将确定性 ID 映射为模糊描述，同时保持**逻辑唯一性**。以 τ^2-bench Retail 域为例，五类关键元素的模糊化策略：

- **User ID**：模糊化为电子邮件或姓名 + 邮编组合（域策略保证唯一标识）；
- **Order ID**：替换为订单中随机物品子集（验证目标订单是唯一包含该组合的订单）；
- **Item ID**：用商品名或商品 + 属性组合描述（验证上下文内唯一性）；
- **Payment ID**：模糊化为支付类型（信用卡含品牌或末四位），确保唯一标识；
- **Address**：若匹配历史订单或默认地址，描述为 “其他订单中用过的地址” 配合城市名。

模糊化约束 $F = \{f_1, f_2, \ldots, f_m\}$ 迫使智能体推理或查询数据库，同时保持到原始约束 C 的确定性映射。

3.3 轨迹生成（Trajectory Generation）

User Simulator LLM 扮演人类用户角色，与 Agent 进行多轮交互。F 嵌入模拟器提示中，关键设计：模拟器被要求**渐进且自然地揭示** F 中的需求，而非一次性披露所有约束。这增加任务难度并更好模拟真实沟通动态。对话结束时模拟器输出终止标记（`###STOP###`），触发会话结束，智能体完整上下文历史记录为轨迹 τ。

3.4 轨迹验证（Trajectory Verification）

区别于依赖 LLM 评估的先前方法，CoVe 利用显式约束集 C 作为**确定性真值清单**，消除 LLM 幻觉评估。基于规则的验证器 $V(\tau, C)$ 解析 τ 中的工具调用记录，逐一对照 C 中每个约束 c_i。

关键特性：验证器评估**结果是否达成**，而非强制预定义动作序列。例如约束 “取消订单 #W6289991”，验证器接受任何导致该订单被取消的工具调用或组合（直接取消或取消所有关联商品），只要在域策略下等价。同时，验证器识别并惩罚不对应任何约束的**冗余操作**。最终综合评分基于：约束满足率（越高越好）和冗余动作数（越低越好）。

3.5 后训练：SFT 与 RL

SFT：使用教师模型作为 Agent 与 User Simulator 交互，生成大量候选轨迹。完成后验证器计算 $S_\tau = V(\tau, C)$，仅保留满分（$S_\tau = 1$）轨迹——即满足 C 中所有约束且无冗余操作的轨迹。此过程生成仅包含正确执行路径的干净数据集，用于微调目标学生模型。

RL：CoVe 同时提供训练环境（学生 Agent 与 User Simulator 交互探索解路径）和奖励信号（验证器产生的 S_τ 直接反馈给 Agent 指导策略更新）。采用 GRPO 算法，每个 prompt 采样 16 条 rollout。

4. 实验设置

基座模型：Qwen3-4B-Instruct-2507，使用 VeRL 框架训练，两节点各 8 块 80GB GPU。SFT 使用 AdamW 优化器，学习率 1×10^{-6}，全局 batch size 128。RL 采用 GRPO，学习率 1×10^{-6}，batch size 64，SGLang 作为 rollout 引擎，采样温度 1.0，用户/助手最大轮数分别为 32/40。

评估基准：τ^2-bench 的 Airline 和 Retail 域，使用 $pass^k$（$k = 1$ 到 4）指标，衡量模型在 k 次连续独立运行中全部成功的概率，评估稳定性与一致性。

数据生成：沙盒数据库由 LLM 和手工规则生成（避免数据污染）。SFT 使用多个 User Simulator（Qwen3-235B、Gemini-3-Pro）增加多样性，教师 Agent 为 Qwen3-235B。RL 阶段仅使用 Qwen3-235B 作为 User Simulator。

5. 主要实验结果

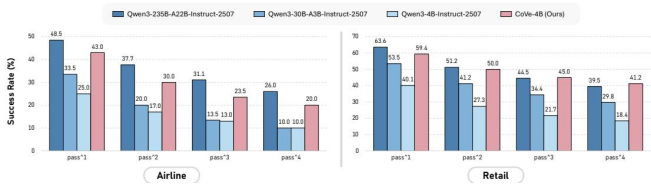


图 2：*CoVe-4B* 在 τ^2*-bench Airline* 和 *Retail* 域的 $pass^1$ 到 $pass^4$ 性能。

模型	Airline	Retail	平均
GPT-5	62.5%	81.6%	72.1%
GPT-4o	47.6%	64.0%	55.8%
Qwen3-235B-A22B	48.5%	63.6%	56.1%
xLAM-2-70b-fc-r	43.3%	59.7%	51.5%
Simia-Tau-32B	48.5%	62.5%	55.5%
xLAM-2-32b-fc-r	47.0%	52.1%	49.5%
Simia-Tau-RL-8B	43.0%	52.4%	47.7%
xLAM-2-8b-fc-r	33.5%	48.9%	41.2%
Qwen3-4B-Inst.	25.0%	40.1%	32.6%
CoVe-4B (Ours)	**43.0%**	**59.4%**	**51.2%**

≤8B 组内领先：CoVe-4B 平均 $pass^1$ 达 51.2%，超越 Simia-Tau-RL-8B（47.7%）和 xLAM-2-8b-fc-r（41.2%）。相比基座

Qwen3-4B-Instruct，绝对提升 **+18.6%**（32.6% → 51.2%）。**跨规模竞争力：** CoVe-4B（51.2%）超越 xLAM-2-32b-fc-r（49.5%）和 Qwen3-30B（43.5%），逼近 xLAM-2-70b-fc-r（51.5%，**17× 参数量**）。与 Qwen3-235B 和 GPT-4o 的差距仅 4.9% 和 4.6%。Retail 域表现尤为突出，$pass^1$ 到 $pass^4$ 均领先同组。

6. 消融实验与分析

6.1 数据质量与规模

训练集	Airline	Retail	平均
无 (基座)	25.0%	40.1%	32.6%
APIGen-MT-5K	34.0%	49.3%	41.7%
Simia-5K	33.0%	46.3%	39.7%
Simia-90K	42.0%	46.5%	44.3%
CoVe-5K	38.8%	50.5%	44.7%
CoVe-12K	**43.0%**	**59.4%**	**51.2%**

关键发现：(1) 控制数据量为 5K 时，CoVe-5K（44.7%）显著优于 APIGen-MT-5K（41.7%）和 Simia-5K（39.7%），证明确定性约束采样与严格验证产生更干净有效的训练信号；（2）CoVe-5K 仅用 Simia-90K **5.5% 的数据量**即达到略高分数（44.7% vs 44.3%），证明零冗余轨迹远优于简单扩大噪声数据；（3）扩展至 CoVe-12K 带来大幅提升，平均达 51.2%。

6.2 训练范式对比

训练方式	Airline	Retail	平均
基座	25.0%	40.1%	32.6%
SFT	**43.0%**	**59.4%**	**51.2%**
RL	32.5%	48.9%	40.7%
SFT+RL	38.0%	55.7%	46.9%

纯 SFT 和纯 RL 均大幅超越基座，验证确定性验证机制作为离线数据过滤器和在线奖励信号均有效。但 SFT+RL 反而低于纯 SFT（46.9% vs 51.2%）。原因分析：SFT 数据由多个强模型（含 Gemini-3-Pro）生成，多样性丰富；RL 在线交互受限于单一较弱模拟器（Qwen3-235B），已优秀的 SFT 模型在更窄环境中交互产生**负正则化效应**，过拟合模拟器特性并遗忘 SFT 获得的泛化能力。

6.3 合成过程动态分析

不同 User Simulator 的轨迹生成成功率差异显著：Gemini-3-Pro 平均产出率 74.0%（Airline 62.8%，Retail 85.1%），Qwen3-235B 仅 38.7%（Airline 27.2%，Retail 50.2%）。Gemini-3-Pro 凭借更强的指令遵循能力，能准确把握对话终止信号时机；较弱模拟器常将 Agent 的澄清请求误判为任务完成而过早终止对话。Retail 域生成成功率一致高于 Airline 域，与 Airline 任务固有的更高难度一致。

7. 与相关工作的对比

vs. APIGen-MT：采用“蓝图到对话”流水线合成多轮轨迹并进行质量验证，但验证依赖 LLM，无法保证确定性正确。CoVe 的约束采样保证可解性，规则验证消除幻觉。

vs. Simia：利用 LLM 模拟环境和用户反馈，支持 SFT 和 RL，但同样依赖 LLM 验证。实验证明 CoVe-5K 以 5.5% 的数据量匹配 Simia-90K 性能。

vs. GEM：从大规模文本语料挖掘隐式多步流程并转化为可执行轨迹，方法正交但验证可靠性不如约束引导。GEM-8B 仅达 33.3%，远低于 CoVe-4B 的 51.2%。

vs. MUA-RL：在 RL 中集成动态用户模拟，但缺乏确定性验证机制。

8. 局限性与未来方向

- **SFT+RL 退化**：当前在线 RL 受限于单一开源模拟器的能力差距。未来方向：（1）采用更强通用模型作为 User Simulator；（2）利用已收集的上下文数据训练专用模拟器；（3）通过 prompt engineering 优化模拟器的对话终止时机判断。
- **域覆盖有限**：当前仅验证 τ^2-bench 的 Airline 和 Retail 域。未来将扩展至 Telecom 域及 BFCL 等其他多轮交互基准，全面验证框架的泛化能力。
- **约束设计依赖领域知识**：模糊化策略和验证规则需针对具体域设计，自动化程度有待提升。

9. 总结

CoVe 通过显式任务约束锚定生成过程，成功弥合现实用户请求的模糊性与工具确定性执行需求之间的鸿沟。约束模糊化激发复杂多轮对话，原始约束作为确定性清单确保验证绝对正确。核心优势：（1）数据效率极高——5K 样本匹配 90K 噪声数据；（2）4B 紧凑模型达到 17× 参数量模型的竞争力水平；（3）框架天然支持 SFT 和 RL 两种后训练范式。开源模型、代码和 12K 高质量轨迹数据集。

MMR-Life：面向真实生活场景的多模态多图推理基准

Li*, Huang*, Jin†, Zhang, Cao, Chen, Liu, Zhao† — 中国科学院自动化研究所 / 中国科学院大学, 2025

1. 动机与问题

近年来，多模态大语言模型（MLLMs）在科学分析和数学推理等复杂任务上取得了显著进展，但其在**真实生活场景**中的多样化推理能力仍缺乏系统评估。现有基准存在两大偏差：

- **任务设计偏差**：知识密集型基准（MMMU、GPQA）依赖专家级领域知识，符号化谜题基准（VisualPuzzles、MME-Reasoning）使用合成图像——两者均偏离日常推理需求；
- **输入形式偏差**：大量基准仅支持单图输入（平均 1–2 张），而真实世界中视觉信息通常以**多图序列**呈现；现有多图基准要么包含非推理任务，要么仅覆盖有限推理类型。

为此，作者提出 **MMR-Life**——首个全面评估 MLLM 在真实生活场景下多模态多图推理能力的基准，包含 **2,646** 道多选题、基于 **19,108** 张主要来源于真实场景的图片，覆盖 **7 种推理类型**和 **21 个任务**。所有问题不依赖专业知识，要求模型从多张真实图片中整合信息并进行推理。

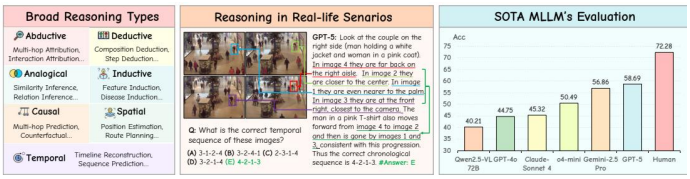


图 1：*MMR-Life* 概览。左：7 种推理类型与 21 个任务。中：典型多图推理示例。右：人类与 *SOTA MLLM* 的性能差距。

2. 基准构建

2.1 七种推理类型

MMR-Life 系统覆盖日常情境所需的推理能力：

- **溯因推理（Abductive）**：307 题，从观察到的事件推断最可能的原因；
- **类比推理（Analogical）**：568 题，通过已知案例的相似性推断新情境；
- **因果推理（Causal）**：263 题，基于原因推断结果；
- **演绎推理（Deductive）**：282 题，从一般规则推导特定结论；
- **归纳推理（Inductive）**：429 题，从具体观察中归纳规律；
- **空间推理（Spatial）**：255 题，理解物体的位置、运动和空间关系；
- **时序推理（Temporal）**：542 题，推理事件的顺序、持续时间和时间关系。

每题平均关联 **7.22 张图片**，远超现有基准；54.95% 为文本选项，45.05% 为图像选项。

2.2 数据构建流程

数据收集来自四类来源：公开图像数据集（Kaggle）、开放网络资源（如 eBird）、公开视频源（提取帧）、以及已有基准。**问答生成**采用自动合成与人工标注相结合——当图片本身包含足够显式信息（如时序帧）时自动生成，需推理隐式信息时人工标注。**干扰选项**生成方面，图像选项通过启发式采样，文本选项调用 GPT-5-mini、GPT-4o 和 Qwen2.5-VL-32B 生成后人工筛选最佳 4 个错误选项。

质量控制三步过滤：（1）难度过滤：用 Qwen2.5-VL-7B、Gemma3-4B、InternVL3.5-8B 三个小模型作答，全部正确的题目视为过于简单而剔除；（2）格式过滤：修正选项间的显著格式差异（如长度），防止模型利用捷径；（3）质量过滤：剔除语义歧义、多正确答案或需专业知识的题目。最终从 3.2K 题筛选为 2,646 题。

2.3 与现有基准的关键区别

MMR-Life 是唯一同时满足以下条件的基准：（1）**纯自然图像**（排除图表和符号图像）；（2）**全新标注**（降低数据污染风险）；（3）**7 种推理类型全覆盖**；（4）**低知识依赖**；（5）**高平均图片数**（7.22 张/题 vs. 现有最高 6.64 张）。

3. 主要实验结果

评估 **37 个先进 MLLM**,包括闭源思考模型(GPT-5、Gemini-2.5-Pro/Flash、o4-mini、Claude-Sonnet-4 等)、闭源非思考模型（GPT-4.1、GPT-4o、Claude-3.7-Sonnet 等）、开源思考模型（VL-Rethinker-72B、QVQ-72B 等）和开源非思考模型（Qwen2.5-VL 系列、Gemma3 系列等）。人类基线由 12 名不同学历背景学生在 210 题子集上完成。

模型	Abd	Ana	Cau	Ded	Ind	Spa	Tem	Avg
Human	79.8	57.7	75.0	70.6	63.4	79.8	79.8	72.3
GPT-5	53.8	78.9	41.1	80.1	78.3	17.3	41.7	**58.7**
Gemini-2.5-Pro	54.4	73.8	37.0	79.4	73.7	25.1	35.8	56.9
o4-mini	41.4	73.6	27.4	71.3	68.1	19.2	32.7	50.5
GPT-4.1	44.3	71.3	22.4	67.4	70.2	13.7	27.3	48.2
Qwen2.5-72B	35.5	55.5	35.4	52.1	55.5	12.9	23.8	40.2
VL-Rethink-72B	36.5	50.9	33.1	56.0	57.6	15.7	21.6	39.7

核心发现 1：MMR-Life 极具挑战性。即使最强的 GPT-5 也仅达 58.69%，与人类 72.28% 相差 **14%**。几乎所有开源模型准确率低于 40%，部分模型（Skywork-R1V-38B：14.13%、InternVL3.5-8B：18.67%）甚至低于随机猜测（20%）。

核心发现 2：推理类型间差异巨大。模型在类比、演绎和归纳推理上表现尚可（GPT-5 达 78–80%），但在**空间推理**上全面崩溃——最高仅 25.10%（人类 79.76%）。因果和时序推理同样是显著瓶颈（最高 41% 和 42%）。模型易于通过特征关联习得类比和演绎能力，但难以学习空间和时序等抽象世界表示。

核心发现 3：开源思考模型收益有限。闭源思考模型普遍优于非思考模型，但开源思考模型（均值 27.15%）反而**低于**非思考模型（均值 29.01%），表明开源思考模型在真实场景泛化上有巨大改进空间。

4. 思考范式分析

4.1 更长思考是否总更好?

对 14 个模型的 semi-log 分析表明，推理性能与思考长度呈**对数线性**关系。但细粒度分析揭示：**长思考并非对所有推理类型有效**。对非思考模型，归纳推理使用 CoT 后性能下降；对思考模型，增加推理预算也未提升归纳推理表现。相反，类比推理从长 CoT 中获益显著。假设原因是长 CoT 仅适合需要逐步推理的任务，归纳推理可能更适合快速思考。

4.2 是否存在通用推理增强方法?

比较 CoT、Self-Consistency（SC@8）、Best-of-N（BoN@8）和 GRPO 四种方法在 Qwen2.5-VL-7B/32B/72B 上的表现：

- **增强方法在大模型上失效**：从 7B 到 72B，其他方法相对 CoT 的增益持续缩减。在 72B 上，BoN（−0.41）和 GRPO（−0.53）均低于 CoT 基线。原因推测是大模型本身采样到正确路径的概率更高，增强方法的边际收益递减；
- **RL 泛化弱于 BoN**：在所有三个模型尺度上，GRPO 的泛化性均弱于 BoN@8。进一步在更多小模型上验证（不同架构和训练数据），RL 训练模型一致性地不如对应 base 模型的 BoN 推理，有时甚至低于 base + CoT。这对 RL 技术提出质疑：小模型上的 RL 是否仅导致对特定数据集的过拟合?

4.3 推理类型间的相关性与模式聚类

计算所有模型在各推理类型上的 Pearson 相关系数，发现：归纳与类比推理高度相关（0.97），两者共享从具体特征中抽象一般规则的模式；空间推理与所有其他类型相关性最低（与归纳仅 0.40），表明空间能力（位置估计、距离感知）难以从非空间任务迁移。层次聚类揭示了**高阶推理模式**的存在：类比-归纳聚为一簇，因果-溯因聚为一簇，空间独立于所有类型。

5. 错误分析

对 GPT-5 和 Gemini-2.5-Pro 各随机采样每种推理类型 20 个错误案例（共 140 例/模型），归纳五类错误：

- **推理错误**（32%，最主要）：包括因果倒置（24%）、时序混淆（42%）、遗漏关键步骤（24%）；
- **抽象错误**（17%）：反映模型短期思维能力不足，如联想

能力弱；
- **知识错误**（17%）：无法调用正确的常识知识进行推理；
- **感知错误**（12%）：无法识别物体的静态属性（颜色、形状）和动态变化（运动）；
- 其余为格式和其他错误。

6. 讨论与局限

- **核心贡献**：首个全面覆盖 7 种推理类型的真实生活多图推理基准；对 37 个 MLLM 的系统评估揭示空间/时序/因果推理瓶颈；深入分析思考范式揭示长思考局限、RL 泛化不足和推理模式聚类；
- **关键启示**：未来模型训练应更关注真实场景推理（尤其空间和时序）；思考预算应按推理类型自适应分配；小模型 RL 训练需要更强的泛化机制；
- **局限**：多选题格式可能无法完全捕捉开放式推理能力；部分推理类型间边界模糊（如溯因与因果）；人类基线仅基于 210 题子集；数据源多样性仍可进一步扩展。

OmniLottie：基于参数化 Lottie Token 的矢量动画生成

Yang[1], Cheng[2], Chen[3], Fu[2,4], Zeng[2], Cai[4], Yu[2], Ma[1] — [1]Fudan, [2]StepFun, [3]HKU, [4]UQ, 2025

1. 动机与问题定义

矢量动画相比光栅视频具有**轻量、可编辑、分辨率无关**的优势，广泛应用于 UI/UX 设计和数字出版。然而现有矢量动画生成方法存在根本性限制：

- **运动先验依赖**：现有方法（AniClipart、LiveSketch 等）依赖参考 GIF 或文生视频模型提供运动先验，需预先手工设计静态矢量图形，无法端到端生成；
- **外观与运动解耦**：优化方法将外观设计与运动生成分离，仅对选定元素施加效果，无法实现完全矢量化的端到端动画合成；
- **Lottie JSON 冗余**：Lottie 是广泛使用的矢量动画格式（单个 JSON 文件存储形状、效果和运动参数），但原始 JSON 包含大量不变的结构化元数据和格式标记，直接让 LLM/VLM 生成原始 JSON 效率极低——现有模型（DeepSeek、GPT-5、Qwen2.5-VL）的生成成功率仅 0–12.7%（表 1）。

OmniLottie 是首个**端到端多模态矢量动画生成框架**，核心创新：（1）设计 Lottie Tokenizer 将冗余 JSON 压缩为紧凑的命令-参数序列（平均 5.3× 压缩）；（2）基于预训练 VLM（Qwen2.5-VL）构建自回归生成模型，支持文本、图文、视频三种多模态指令输入；（3）构建 MMLottie-2M 大规模数据集（200 万 Lottie 动画配对文本/图像/视频标注）。

2. 系统架构

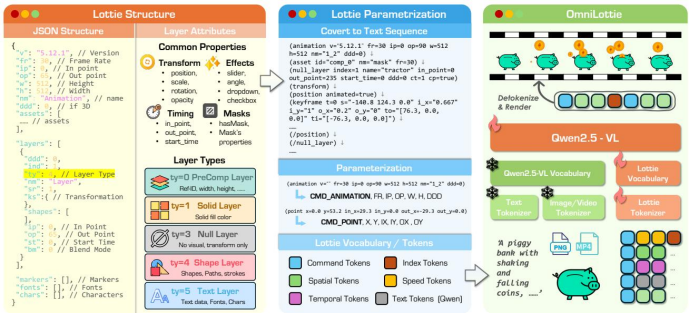


图 1：*OmniLottie* 架构。*Lottie JSON* 经结构重组后由 *Tokenizer* 参数化为命令-参数序列，扩展 *Qwen2.5-VL* 词表后进行自回归生成。

2.1 Lottie 结构重组

原始 Lottie JSON 包含大量冗余格式标记。OmniLottie 首先将其重组为核心属性 $\mathcal{M} = \{v, fr, ip, op, w, h, nm, ddd\}$ 和条件属性（assets、markers、fonts、chars，仅在需要时出现）。支持五种基本图层类型：预合成（ty=0）、纯色（ty=1）、空（ty=3）、形状（ty=4）和文本（ty=5）。每个图层形式化为：

$$\mathcal{L}_i = (\tau_i, \mathcal{A}_{\tau_i}, \mathcal{T}_i, \mathcal{E}_i)$$

其中 τ_i 为图层类型，$\mathcal{A}_{\tau_i}$ 为类型特定属性，$\mathcal{T}_i$ 和 $\mathcal{E}_i$ 分别为变换和效果。通过父子关系重建完整场景层级树，实现 JSON 格式的**无损压缩**。

2.2 Lottie Tokenizer

Tokenizer 将层级化 JSON 转换为紧凑的命令-参数序列。核心是**基于偏移的离散化方案**：

$$\text{token}(x, t) = \lfloor x \cdot s_t \rfloor + o_t$$

其中 x 为参数值，t 为参数类型（时间、空间、变换、样式），s_t 为类型特定缩放因子，o_t 为词表偏移量。不同参数类型占据互不重叠的词表区间，通过数据驱动的分布分析确定各类参数的值域范围。最终 token 序列为：

$$\mathcal{T} = [\mathbf{CMD}_1, p_{1,1}, \ldots, p_{1,k_1}, \mathbf{CMD}_2, \ldots, \mathbf{CMD}_M, p_{M,k_M}]$$

文本类参数（字体、字符串等）使用 Qwen2.5-VL 原生分词器编码，格式为 $[count, tok_1, \ldots, tok_n]$，保留预训练语言理解能力。填充值和结构边界使用特殊标记 token。整个过程**完全可逆**——通过逆变换 $p = (\text{token} - o_t)/s_t$ 确定性重建有效 Lottie JSON。

压缩效率：原始 JSON 平均 2,562 tokens → 结构化文本 884 tokens → 最终命令序列 486 tokens，实现 5.3× 压缩，序列长度减少 81%。

2.3 模型架构与训练

以 **Qwen2.5-VL** 为骨干，引入随机初始化的 Lottie 词表嵌入以表示命令和参数结构。预训练权重为处理交错视频/图像/文本指令提供有效初始化。训练目标为标准交叉熵损失的自回归生成：

$$\theta^* = \arg\min_\theta -\sum_{i=1}^{L} \log P(x_s^{[i]} | x_c; x_s^{[<i]}; \theta)$$

其中 x_c 为多模态指令，$x_s^{[i]}$ 为第 i 个 Lottie token。

3. MMLottie-2M 数据集

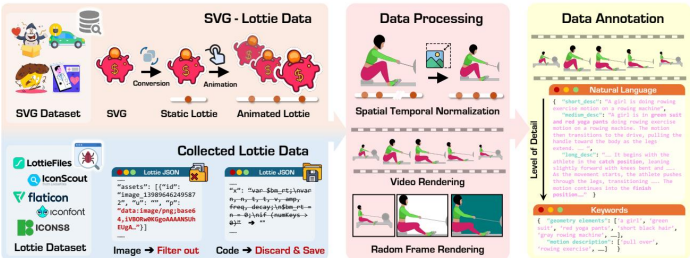


图 2：数据构建流水线。包含数据收集与转换、处理（时空归一化）、标注三个阶段。

3.1 数据收集与处理

Lottie 数据策展：从 LottieFiles、IconScout、Flaticon、Iconfont、Icons8 五大平台收集约 120 万 Lottie 文件。清洗移除 base64 图像层、非视觉层（音频/摄像机）和 After Effects 表达式。

SVG-Lottie 数据策展：基于 OmniSVG 大规模 SVG 集合，将静态 SVG 转换为 Lottie 并施加随机动画效果（平移、缩放、旋转、透明度变化及组合运动），生成约 80 万额外 Lottie 动画，解耦内容与运动语义。

运动库数据增强：分析 100 万原生 Lottie 文件，提取变换轨迹生成运动签名，通过语义聚类提取典型运动模板，注入静态 SVG-Lottie 文件，大幅缩小路径分布差距并拓宽动画层覆盖。

时空归一化：空间归一化至 512 × 512 画布（居中对齐保持宽高比），时间归一化至 0–60 时间戳范围。

数据标注：使用 Qwen2.5-VL 采用**粗到精策略**——先生成一句话整体描述（主体、运动、颜色、风格），再逐帧提供时序细节（“begins with”、“then” 等连接词）。

3.2 MMLottie-Bench 评估协议

包含 **Real Subset**（450 个专业设计师作品，严格与训练集不重叠）和 **Synthetic Subset**（由 GPT-4o/Gemini3.1-Pro/Seedance 合成，防止未来数据污染）。每个子集各 150 个样本覆盖三个任务。评估维度：视觉质量（FVD）、语义对齐（CLIP、Object Alignment 0–10、Motion Alignment 0–10），后两者使用 Claude-3.5-Sonnet 作为 LLM 评判。

4. 核心实验结果

任务	方法	成功率	FVD↓	Motion↑
4*T2L	DeepSeekV3	9.3%	671.8	2.09
	GPT-5	12.7%	715.7	0.71
	Recraft	77.3%	300.7	4.68
	Ours	**88.3%**	**202.1**	**5.94**
4*TI2L	GPT-5	28.0%	546.7	0.95
	AniClipart	87.3%	266.5	3.47
	Livesketch	91.3%	868.2	2.42
	Ours	**93.3%**	**180.3**	**4.44**
3*V2L	GPT-5	9.2%	639.1	–
	Gemini3.1	0.0%	1076.2	–
	Ours	**88.1%**	**227.1**	–

Text-to-Lottie：OmniLottie 成功率 88.3%，FVD 202.14（最优），Motion Alignment 5.94（最优）。Qwen2.5-VL(3B)

成功率 0%，GPT-5 仅 12.7%。商业工具 Recraft 虽有竞争力的 CLIP 分数（0.295），但运动保真度和 token 效率远不如 OmniLottie。

Text-Image-to-Lottie：OmniLottie FVD 180.27、Object Alignment 5.10、Motion Alignment 4.44，均为最优。AniClipart 和 Livesketch 虽成功率尚可，但运行时间极长（分别 1212s 和 723s vs. OmniLottie 88.6s），且无法输出原生矢量格式。

Video-to-Lottie：OmniLottie 实现最优 FVD（227.11）、PSNR（16.08）、SSIM（0.82）和 DINO（0.92）。GPT-5 成功率仅 9.2%，Gemini3.1-Pro 成功率为 0%。OmniLottie 生成显著更丰富的 token 序列（36.8k vs. 基线 ∼1k），支持更具表现力的矢量动画生成。

效率对比：OmniLottie 每次成功生成约 31s，而 LiveSketch 需 1,625s（52× 慢），AniClipart 需 16,438s（530× 慢）。

5. 消融实验

5.1 SVG 数据混合比例

测试四种配置：纯 Lottie、纯 SVG、高 SVG 比例（50%）、适度 SVG 比例（30%）。结果：**30% SVG 混合**达到最优平衡——SVG 提供多样几何形状增强几何理解，Lottie 提供复杂运动。过多 SVG 偏向简单运动，Motion Alignment 从 5.63 降至 2.35（纯 SVG）。最优配置 FVD：269.50（T2L）、359.56（TI2L），显著优于其他配置。

5.2 Lottie Tokenizer 必要性

对比三种配置：预训练 Qwen2.5-VL（直接生成）、微调于原始 JSON、使用 Tokenizer 微调。结果：

- 预训练 VLM：三个任务成功率均为 0%；
- 微调原始 JSON：成功率 13.4%/15.9%/10.2%，FVD 459/476/431；
- **使用 Tokenizer**：成功率 **97.3%/92.0%/90.7%**，FVD **269/360/282**。

Tokenizer 在所有任务和指标上均带来显著提升，验证其结构化紧凑表示对生成质量和计算效率的关键作用。Video-to-Lottie 任务中 PSNR 从 17.23 提升至 19.05，DINO 从 0.68 提升至 0.88。

6. 失败模式分析

论文建立五级失败分类法：Level 1（规范违反）、Level 2（结构性失败）、Level 3（渲染失败）、Level 4（管线失败）、Level 5（输入依赖失败）。

OmniLottie 主要为 Level 2–3 失败：空图层生成（∼35%）和渲染级失败（∼65%，含缺失样式属性 40%、时间可见性错误 25%、透明度/缩放值错误 25%、画布外定位 10%）。失败率随输入复杂度递增：T2L 2.7% < TI2L 8.0% < V2L 9.3%。

基线模型失败层级更低：Qwen2.5-VL 在 Level 1（模式幻觉，0% 成功）；DeepSeek 在 Level 1（属性幻觉，29.3%）；AniClipart 在 Level 5（缺乏语义关键点，7.3%）。

7. 用户研究

20 名参与者对 900 个评估案例（300/任务）在视觉质量、条件遵循、动画质量、几何保真度四个维度进行 5 星排名。OmniLottie 在所有维度和任务中均获最高平均排名。相关性分析表明 Object Alignment 和 Motion Alignment 指标与人类判断强正相关（Motion Alignment Pearson $r = 0.4823$，$p < 0.001$），验证了自动化评估指标的有效性。

8. 讨论与局限

核心贡献：首个端到端多模态矢量动画生成框架；Lottie Tokenizer 实现 81% 序列压缩；MMLottie-2M 大规模数据集（已公开）；标准化评估协议 MMLottie-Bench。

主要局限：

- 自回归解码仍可能产生无效序列（成功率未达 100%）；
- 对复杂动画的泛化能力和上下文长度仍受限制；
- Lottie 格式本身无法表示所有矢量动画效果（如复杂 3D 变换）；
- 未来方向：约束解码（保证语法合法性）、基于可渲染性奖励的强化学习、与 After Effects 等专业工具的智能体集成。